

UniResolve

Session: 2022-2026

Project Members:

Muhammad Saddique	FA22-BCS-053
Asad Haroon	FA22-BCS-004
Shahzaib Khurram	FA22-BCS-027

Supervised by: Prof. Dr. Faisal Riaz (Professor)

Department of Computer Science and Information Technology

Faculty of Natural and Applied Sciences

Mirpur University of Science and Technology (MUST)

Fall 2026

Mirpur AJ&K, Pakistan

Declaration

We hereby declare that this document and research are genuine and there are no instances of plagiarism in this thesis report. If discovered, on the other hand, at any point, we will be answerable for the conclusions. It is further declared that we have done this whole work with the report by ourselves with the supervision of our teacher " **Prof. Dr. Faisal Riaz** ". If any part of this work is found to be copied from any source or any project, we will be answerable for the outcomes.

_____

Muhammad Saddique

FA22-BCS-053

_____

Asad Haroon

FA22-BCS-004

_____

Shahzaib Khurram

FA22-BCS-027

Dedication

We want to dedicate this project to our parents and all the people who supported us throughout this journey. Our teachers cheered us and helped us in our difficult situations, especially our supervisor **Prof. Dr. Faisal Riaz** who was so humble and kind to us. Without his support, it could not have been possible to complete our project.

Acknowledgment

First, we are obliged to Allah Almighty, the Merciful, the Beneficent, and the source of all Knowledge, for granting us the courage and knowledge to complete this project.

It was difficult to complete this project without the help of many people. Many thanks to our supervisor **Prof. Dr. Faisal Riaz**. He helped us understand the confusion and guided us through every phase of development. And we are thankful to our parents and friends for their support and prayers. Special thanks to our beloved and respectful teachers who encouraged us frequently.

Muhammad Saddique
FA22-BCS-053

Asad Haroon
FA22-BCS-004

Shahzaib Khurram
FA22-BCS-027

Abstract

Universities face increasing difficulty in managing students' complaints due to reliance on manual or semi-digital methods that lead to delays, poor tracking, and lack of transparency. Students are sometimes not aware of the resolution of their complaints, and administrative staff is sometimes in on the process of settling their complaints. Have difficulties with dealing with volumes of issues effectively. These restrictions are all evidence of the need for An integrated and automated solution that provides communication, accountability and timely resolution of complaints. This project introduces a complaint management system called UniResolve that is implemented as a web-based application. Complimentary Complaints and Taking Action: digitise and streamline complaint lifecycle within Mirpur University of Science and Technology. Technology (MUST). The system is designed to enable students to submit individual and/or collective answers to questions. Complaints can be made, supporting documents attached and progress monitored using a special tracking ID. It Assigns access for staff, students, and administrators and provides automatic access A complaint routing, priority management, a complaint forwarding system and an in-app notification. A student comments/feedback system, email notifications through an asynchronous queue and a system, Withdrawal of complaints, profile management of students and a public tracking page. Built using PHP 8.2, MySQL and Tailwind CSS v4, UniResolve is designed to cut down on response time and enhance, transparency, and an efficient, scalable and reliable complaint-handling solution. in universities.

Project Brief

Field	Details
Project Title	UniResolve
Objectives	To create a system that is `centralised'. allows efficient submission or tracking of, And complaints handling of students. It seeks to automate the process of routing complaints, and speed up the response time and enhance transparency and accountability within the university.
Developed by	Muhammad Saddique (053), Shahzaib Khurram (027), Asad Haroon (004) BS (CS) (Session 2022-2026)
Supervised by	Prof. Dr. Faisal Riaz (Professor)
Technology used	Visual Studio Code, PHP 8.2, MySQL/MariaDB, Tailwind CSS v4, XAMPP, PHPMailer
System used	Intel Core i5, 8GB RAM and 256 SSD
Operating System used	Windows 11

Key Terms

Term	Description
Visual Studio Code	Integrated development environment (IDE)
CMS	Complaint Management System
HTML, CSS, JS / PHP	Primary programming languages
UI / UX	User Interface / User Experience
PDO	PHP Data Objects – Database access layer
MySQL / MariaDB	Relational Database Management System
Tailwind CSS	Utility-first CSS framework for styling
RBAC	Role-Based Access Control
CRUD	Create, Read, Update, Delete operations
Tracking ID	Unique identifier (e.g. CMP-2026-CS-A3B2F1) for each complaint
bcrypt	Password hashing algorithm for secure authentication
XAMPP	Apache, MySQL, PHP development environment
PHPMailer	PHP library for sending emails via SMTP
Prepared Statements	Parameterized SQL queries to prevent SQL injection
.env	Environment configuration file for sensitive credentials

Table of Contents

Declaration 2

Dedication 3

Acknowledgment 4

Abstract 5

Project Brief 6

Key Terms 7

Table of Contents 8

Chapter 1 13

Introduction 13

1.1 Project Background 14

1.2 Existing System / Description of the Current Situation 14

1.3 Problem Statement 15

1.4 Proposed Solution 15

1.5 System Architecture 15

1.6 Motivation and Challenges 17

1.7 Goals and Objectives 18

1.8 Scope 19

1.8.1 Student Module 19

1.8.2 Staff Module 19

1.8.3 Admin Module 19

1.9 Expected Outcome 19

1.10 Summary 20

Chapter 2 21

Application Survey 21

2.1 Existing System / Current Situation 22
2.2 Limitations of the Current Manual System 22
2.3 Summary 23
Chapter 3 Requirement Analysis 24
3.1 System Requirements 25
3.2 Functional Requirements 25
3.2.1 User Authentication and Access Control 25
3.2.2 Role-Based Access Control (RBAC) 25
3.2.3 Complaint Submission and Tracking 26
3.2.4 Complaint Routing and Assignment 26
3.2.5 Complaint Management (Staff) 27
3.2.6 Notifications and Email System 27
3.2.7 Admin Panel Functions 27
3.3 Non-Functional Requirements 28
3.3.1 Performance Requirements 28
3.3.2 Usability Requirements 28
3.3.3 Security Requirements 28
3.3.4 Reliability Requirements 29
3.3.5 Scalability Requirements 29
3.3.6 Maintainability Requirements 29
3.4 Overall Description 30
3.4.1 User Needs 30
3.4.2 Product Perspective 30
3.4.3 Product Function 30
3.4.4 Operating Environment 31

3.5 Process Model 32
3.5.1 Why use the Spiral Model 32
3.5.2 Spiral Model Phases 33
3.6 Summary 34
Chapter 4 35
Feasibility Study 35
4.1 Feasibility Study 36
4.2 Technical Feasibility 36
4.3 Economic Feasibility 37
4.4 Operational Feasibility 37
4.5 Legal Feasibility 37
4.6 Tools and Technologies 38
4.6.1 PHP 8.2 38
4.6.2 MySQL / MariaDB 38
4.6.3 Tailwind CSS v4 39
4.6.4 PHPMailer 39
4.6.5 XAMPP & Visual Studio Code 40
4.6.6 UI/UX Design 40
4.7 Summary 40
Chapter 5 42
System Design 42
5.1 Introduction 43
5.2 Use Case Diagrams 43
5.2.1 Student Use Case 43
5.2.2 Staff Use Case 44

5.2.3 Admin Use Case 45
5.3 Use Case Description 47
5.3.1 Use Case Table for Login 47
5.3.2 Use Case Table for Submit Complaint 48
5.3.3 Use Case Table for Add Comment 49
5.3.4 Use Case Table for Withdraw Complaint 50
5.3.5 Use Case Table for Update Status 51
5.3.6 Use Case Table for Manage Roles 52
5.5 Entity Relationship Diagram (ERD) 53
5.5.1 ERD Entity Descriptions 54
5.5.2 ERD Relationships 55
5.6 Summary 56
Chapter 6 Implementation & Testing 57
6.1 Features and Results 58
6.1.1 Login Page 58
6.1.2 Student Dashboard 59
6.1.3 Submit Complaint Page 59
6.1.4 Track Complaint & Comment System 60
6.1.5 Staff Dashboard & Assigned Complaints 61
6.1.6 Staff View Complaint Page 61
6.1.7 Admin Dashboard & Analytics 62
6.1.8 Manage Roles & Users Page 63
6.1.9 Manage Routing Rules Page 64
6.2 Testing 64
6.2.1 Black Box Testing 64

6.2.2 White Box Testing 65
6.3 Black Box Test Cases 65
6.3.1 Test Case 1: Login & Authentication 65
6.3.2 Test Case 2: Submit Complaint & Auto-Routing 65
6.3.3 Test Case 3: Notifications & Comments 66
6.3.4 Test Case 4: Withdraw Complaint 66
6.3.5 Test Case 5: Update Complaint Status 67
6.3.6 Test Case 6: Role Management (Admin) 67
Chapter 7 68
Conclusion and Future Enhancement 68
7.1 Introduction 69
7.2 Results and Discussion 69
7.3 Future Enhancements 70
7.4 Summary 71
Appendix A: A step-by-step guide to use this application 75
1. Accessing the Application 75
2. Submitting a Complaint (Student) 75
3. Tracking and Communication 75
4. Staff and Admin Operations 75
Appendix B: Code Snippets 76
Appendix C: About Developers and Supervisor 77
About Supervisors 78
About Developers 80
..... 80

Chapter 1

Introduction

1.1 Project Background

A university that continues to expand, increases the number of students as well as their daily life challenges. From the academic aspect, to hostel facilities, transport or internet aspects, students have a lot of trouble getting their complaints addressed and resolved in a timely fashion. There is no formal means of reporting or monitoring complaints at present, it's all by email, phone and paper. This results in delays, loss of requests and a lot of frustration for both. parties. No central system, delays, mismanagement and lack of transparency. Administrative staff may not be aware of pupils' complaints and whether action is taken or not. may not be able to remember and/or prioritize complaints. The department's staff members are also overloaded with the volume of complaints, and have to deal with them on a manual basis With no system of prioritization or tracking. So we've developed UniResolve: an easy, clear and effective online service to submit Make a complaint, monitor progress and get status reports, all from one place. The system Confirms complaints are directed to the proper departments and retains the history of complaints. all actions taken such as status changes, forwarding, priority changes etc. This system contains a well-structured database of entities such as Users, Students, Staff, Complainant, etc.It includes a well-structured database containing entities like Users, Students, Staff, Complainant, etc. Complainant Type, Routing Rules, Departments, Complainant Status History, Forwarding Logs, Priority Records, Notes, Notes by Internal Users, Notifications and Comments. The structured database allows for effective complaint management, role-based access and communication with stakeholders.

1.2 Existing System / Description of the Current Situation

As it is now, any grievance by a student may be expressed through an e-mail, a telephone call, or a paper form. There is no central system to keep track of these complaints. Consequently, requests may be forgotten or lost or not be followed up. Students are not able to determine whether complaints have been received or when they will be addressed.

Staff in the department are also overworked, having to handle complaints manually without anything to prioritize or track. In general, the process is ineffective, anonymous and unaccountable.

1.3 Problem Statement

Poor communication between students and university departments due to the absence of a centralized, electronic complaint management system at MUST. No clear accountability, delays, and loss of records due to manual handling. Pupils are not listened to and complaints are not dealt with effectively by staff. The administration also has no way to gauge performance or problems. This creates a gap which affects students' satisfaction and effectiveness in operation.

1.4 Proposed Solution

UniResolve is a complaint management system for MUST that is a web-based solution. It Enables pupils to log-in, make category complaints and get a unique ticket ID for their complaint tracking purposes. Complaints will automatically be referred to the appropriate department. The appropriate individual and the relevant person. Staff will be able to view, update and comment on the complaints The monitoring, reporting and management will be possible on their dashboards and Admins. system configuration. System provides you with an alert of what is happening and it responds very quickly by email. By all means notification.

1.5 System Architecture

UniResolve is based on a multi-layered technology to guarantee secure, scalable and efficient Handling complaints in the university.

Users:

The system has three users - students, staff in the department and administration. Students can make complaints and view their status, staff can manage complaints and get them Administer users, departments and system reports which are resolved.

Web Application:

User friendly Web application, with secure access for all users. It processes user Logins, complaints, viewing the dashboard and interacts with backend.

Backend Services:

The back-end features complaint management, automated complaint reply generation, authentication and authorization. complaint assignment and email notification services. These services will be available to all. They are supported in a secure manner, with a proper complaint routing and real-time updates.

Database:

All information related to users, complaints, departments etc. is stored in the system based on the MySQL database. Maintains a secure history of the status of information, notifications, comments and complaints in the system operation and reporting.

Notification System:

Complaints are sent out automatically to students and staff via the email notification module whenever they are submitted, assigned, forwarded or resolved, ensuring the student knows about the change and the staff member is kept in the loop. A PHPMailer based asynchronous email queue will ensure reliable delivery.

Technology Stack:

The system is designed to be reliable and easily deployable on university servers, using PHP 8.2, HTML, CSS (Tailwind CSS v4), JavaScript, and MySQL/MariaDB.

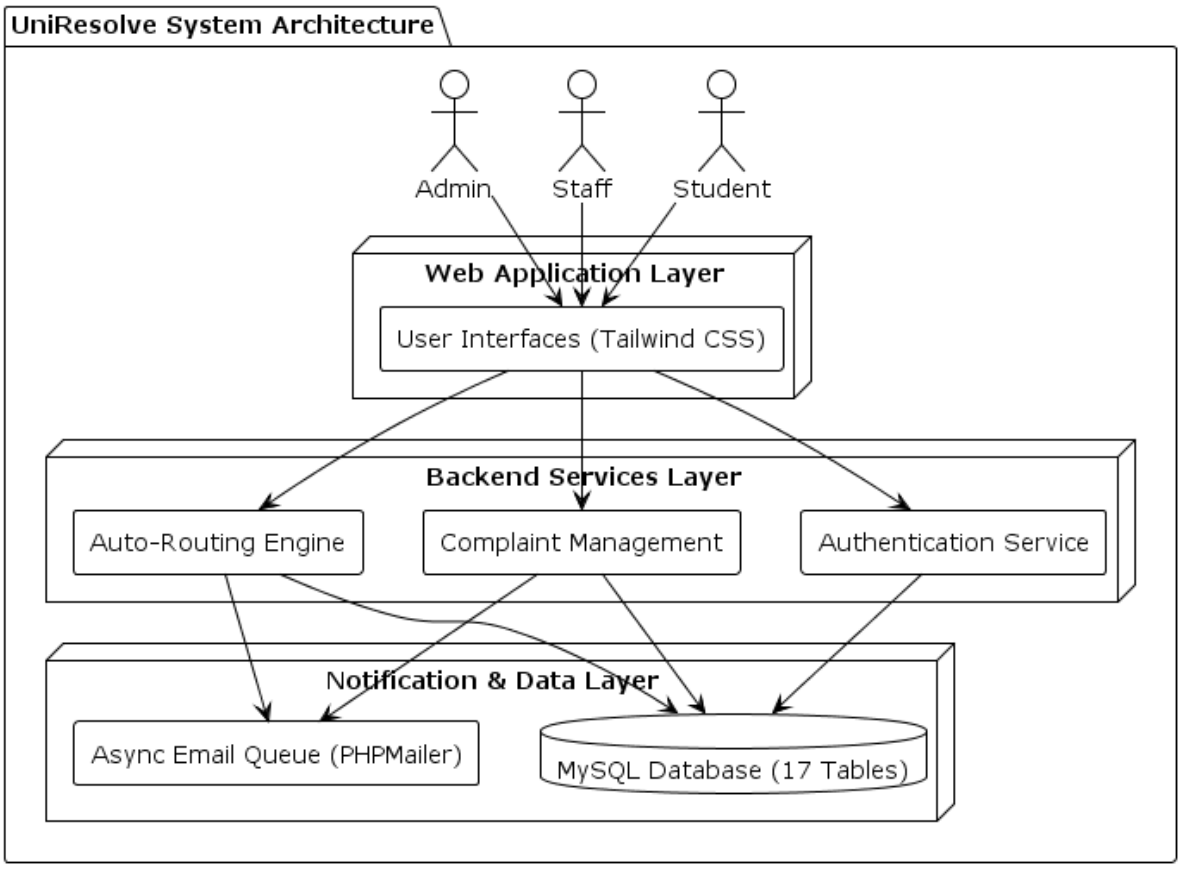


Figure 1.1: System Architecture of UniResolve

1.6 Motivation and Challenges

Motivation

The main idea that inspired UniResolve was that of the inefficiencies found in the current complaint handling system at MUST. Some of the above factors are key motivators for:

- A lack of transparency in complaint procedures.
- There is no effective monitoring system for pupils
- Manual handling contributing to delays and loss of records
- Lack of ability to monitor performance or detect recurring problems for administration.
- Increasing number of complaints with no scalable system

Challenges

Multiple Roles (Student, Staff, Admin) with Dynamic Navigation, Notifications and Features.

- Efficient Complaint Routing – automatically routing complaints to the right staff member according to a set of routing rules, that also specify a response deadline.
- Real-Time Communication – In-app notifications, email queue processing and a comment/feedback system for bi-directional communications.
- Data Security and Privacy – Input Sanitization, Prepared Statements, bcrypt hashing, and using .env for credential management.
- Scalability – ability to deal with growing number of users and complaints using a well-normalized relational database of 17 tables.
- User Adoption – Creating a user friendly, responsive interface with mobile navigation support.

1.7 Goals and Objectives

Project Goal

The primary objective of UniResolve is to create a centralized, transparent and efficient complaint handling system to enhance communication between the University Departments and students.

Objectives

- To create web based complaint management system in PHP & MySQL.
- To facilitate students to file individual/combined complaints with attachments.
- To provide role-based access for both students and staff, along with dynamic dashboards for students and admin
- To automatically assign complaints based on customizable routing rules with response time limits
- To roll out a complete notification system (in-app + email through an async queue).
- To offer a space for giving and receiving feedback and comments (two-way communication between students/teacher)
- Separate letterboxes for the collection of incoming and outgoing mail.Collection of incoming and outgoing mail in separate letterboxes for support of complaint forwarding, priority management, and withdrawal.
- To enable the public to view a complaints tracking page without sign in
- To provide reports and analytics for administrative monitoring

1.8 Scope

1.8.1 Student Module

This module enables students to sign-up and log-in securely. Students can complain in the following categories and departments. They submit individual or collective complaints, attach supporting files and can follow up on complaint status through a unique ticket ID. Students can also make comments, remove pending complaints, handle their profile and get in-app and email alerts for updates on complaints.

1.8.2 Staff Module

You can use the staff module to see the complaints assigned to department staff. Staff members are able to change complaint status, add resolution information, reply to student notes, share/complain with other staff members, adjust priority and add private staff notes. CMS Coordinators can import students in bulk via CSV, and assign roles to students. This module aids the efficient management and prioritization of complaints and holds the Department accountable.

1.8.3 Admin Module

The admin module enables the administration to have complete control of the system. Admins can create, view and manage users, roles, departments, types of complaints and routing rules. They can monitor the status of complaint resolution, view system logs, and view user activity. and create reports. The aim of this module is to ensure the regular operation of the system. transparency.

1.9 Expected Outcome

- An effective online complaints process
- Enhanced communication between students and administration with comments and notifications.
- Automated routing and priority management decrease complaint resolution time
- Unique tracking IDs and public tracking page result in a transparent tracking system.
- Forwarding and priority features for efficient workload distribution among staff.

- Enhanced reporting, analytics, and activity logs to facilitate better decision-making.
- Collective complaints – lets multiple students participate in a single complaint.

1.10 Summary

This chapter presented the system UniResolve which has been developed to address the need The introduction of an overall complaint management system within MUST. It explained the problem, Proposed solution, system architecture, motivation, problem challenges and objectives of the system.

This proposed system will seek to remove the inefficiencies in manual complaint handling and launch an organized, clear and automated platform. UniResolve is developed Is built with PHP 8.2, MariaDB/MySQL and the CSS framework Tailwind CSS v4 and contains 3 users. Access to a dashboard and features such as notifications and custom dashboards depending on their role (Student, Staff, or Admin), Collective complaints, public tracking and comments.

Chapter 2

Application Survey

2.1 Existing System / Current Situation

At this present time, a student who has a complaint at MUST can write an e-mail, call or complete a paper form. There has not been any central system to monitor these complaints. Consequently, requests may go unattended, unanswered or forgotten. Pupils cannot check whether their complaint has been received or when it will be acted upon.

Staff in the department are also inundated, having to deal with complaints on a case-by-case basis and have no tools to prioritize and track progress. In total, it is a slow, unaccountable and untraceable process. Major problems with the existing manual system are:

- No centralised database – Complaints are distributed in emails, telephone logs, physical forms and do not have a centralised database.
- No tracking mechanism – Students are unable to see the status of complaints submitted.
- No accountability – No audit trail tracking who received a complaint, when and how long it took to be acted on.
- No automated routing – Complaints may be delayed as they have to be sent to the proper department/person manually.
- No performance monitoring – Administration has no information on department response time or any recurring issues.
- No notification system – Students are not informed of the progress of their complaint.
- Scalability problems – Manual handling becomes less and less manageable as the number of enrolments increases.

2.2 Limitations of the Current Manual System

Aspect	**Current Manual System**	**UniResolve (Proposed)**
Complaint Submission	Email, phone, paper forms	Online form with categories and attachments
Tracking	None	Unique tracking ID (CMP-2026-CS-XXXXXX)

Status Updates	None – students must follow up manually	Real-time in-app + email notifications
Routing	Manual forwarding by staff	Automatic routing via configurable rules
Accountability	No audit trail	Complete status history and forwarding logs
Priority Management	None	Normal / Medium / High with history
Communication	Separate emails or calls	Built-in comment/feedback system
Reporting	None	Department performance and category analytics
Collective Issues	Individual complaints only	Collective complaints with join feature
Public Access	None	Public tracking page without login

Table 2.1: Comparison of current manual system vs UniResolve

2.3 Summary

This chapter examined the current complaint handling process in MUST, which are completely manual processes (email, phone calls, paper forms). The analysis found key issues, such as the lack of a tracking mechanism, no centralized database, no automated routing, no notification system and no accountability or audit trail.

Each of the limitations identified has been addressed by the digital solution, the UniResolve platform, as illustrated below. The implications of these insights leads us to believe that the need for a complaint management system that is purpose built, transparent and efficient for the university context at MUST is paramount.

Chapter 3

Requirement Analysis

3.1 System Requirements

System Requirements describe the hardware, software, and technical infrastructure that is required to develop and deploy the Web Application. These are the requirements that make the application function properly, that all the functions it should support are available and that it gives a smooth end user and administrator experience.

3.2 Functional Requirements

Functional requirements state the manner in which the system should behave and function in order to satisfy the needs of the system users. These are based on the roles and interaction that is expected in the application. The proposed system has three main users: Students, Staff and Administrators.

3.2.1 User Authentication and Access Control

- The prevention of unauthorized access and the control of access to information and services.
- Admin should have the ability to set up user accounts using email credentials.
- Each user needs to use his/her own login ID and password.
- Login/ logout capabilities should be included with session management.
- If the user is flagged by admin, users should be required to change their password on first login.
- Anti-caching headers should be used to make sure that users cannot still access content after logging out by pressing the back button on their browser.

3.2.2 Role-Based Access Control (RBAC)

The system must provide access rights different for various roles:

- Student: May file individual or collective complaints, can withdraw complaints, can follow up on complaints, can enter comments, can view history, can manage profile, and can get notifications.

- Staff (General): Logs complaints, updates status, forwards complaints, changes priority, adds notes, and replies to student comments.
- Staff (CMS Coordinator): Can carry out all general staff functions and other tools to use as a coordinator for students when importing large amounts of students in CSV format, and for assigning their roles to other staff in the department.
- Staff (Session Advisor): Individual staff member to advise an individual session.
- Admin: May add users, roles, departments, complaint types, routing rules, view all complaints, monitor system activity and create reports.

3.2.3 Complaint Submission and Tracking

A complaint must be submitted and tracked using the following method:

- Students should be able to file complaints with the title, description and category selection.
- Collective complaints to be supported by system, to include students in the same department.
- Attachments of files (jpg, jpeg, png, pdf, doc, docx) are required with each complaint and must be included in the system.
- It is important to note that a unique tracking ID (CMP-Year-Dept-Random) should be auto-generated dynamically for each complaint.
- Students are to be able to withdraw complaints from the “Pending” or “In Progress” status.
- Students should be able to see the complaint from the tracking ID in a public tracking page (no log-in) and/or their dashboard.
- Complete detail of all complaints should be displayed, and should show the status timeline, attachments, and a two-way comment/feedback area.

3.2.4 Complaint Routing and Assignment

- Admin should be able to create Routing rules according to Department and type of Complaint.
- A complaint must be automatically routed to the right staff member based on the routing rules when it is reported.

- The max response hours should be set in the routing rules and response due dates should be set automatically.

3.2.5 Complaint Management (Staff)

- Staff should have access to all complaints that have been assigned to them, and be smartly sorted by priority and status.
- Staff should have the ability to change the status of the complaint from Pending, In Progress, Resolved, Rejected.
- Ability for staff to refer complaints to other members of staff in the same department, with a reason.
- Staff need to have the ability to alter priority of complaint (Normal, Medium, High).
- Staff should be able to make private notes for the staff member to whom they are assigned.
- Staff should be able to communicate with the students through the comment/feedback section.

3.2.6 Notifications and Email System

- The system needs to notify the user of important actions (complaint submitted, status change, new comments, etc., forwarding, withdrawal).
- They should be able to check their notifications in a dropdown in the navigation bar and on a dedicated notification center page, and can "mark all as read".
- Must send email notifications for critical events; it must happen asynchronously, through an email queue system, and must not block the user interface, must use PHPMailer to send the email.

3.2.7 Admin Panel Functions

Admin needs to be able to:

- Manage Roles: Add roles, update role descriptions and administer roles to staff.
- Manage Users: Activate, Deactivate, Create User Accounts (Dynamically) based on User Type.
- Add, view and delete departments.

- Manage Complaint Types: Add, View and Delete Complaint Types.
- Configure auto assignment rules (department + type → staff + response time): Manage Routing Rules.
- View All Complaints: A complete view that includes search, filters, full history and forwarding logs.
- Monitor System Activity: See activity logs with status changes and events forwarded.
- Generate Reports: Analytics by department and complaint category.

3.3 Non-Functional Requirements

Non-functional requirements specify the quality aspects of the system – that is, the way it does things. These requirements guarantee reliability, security, scalability, and usability of the system.

3.3.1 Performance Requirements

- It is essential that the system is able to operate with several users concurrently without delay or failure.
- All significant functions (submitting a complaint, search and filtering) should not take more than 3 seconds even in "normal conditions".
- All database queries need to be optimized through proper indexing and prepared statements.
- Emails have to be sent asynchronously (process_queue.php) for fast UI response times.

3.3.2 Usability Requirements

- The user interface should be user friendly, attractive and easy to operate for all users.
- The application should be responsive and responsive on desktop, tablet and mobile.
- Clear status badges, color coding and modern UI/UX design principles (Tailwind CSS) must be used in the system.

3.3.3 Security Requirements

- All passwords should be securely stored in a bcrypt manner.
- All database queries should be prepared, to avoid SQL injection.

- Input from users should be sanitized using custom functions such as sanitize_input and clean_text to help mitigate XSS (Cross-Site Scripting).
- A session-based authentication should not allow access to protected pages to unauthorized users.
- Role-based access control should not allow users to do things or see pages that they are not authorized to do.
- Any data that will be used in the application's configuration should be kept in the .env file, which is not included in the version control.

3.3.4 Reliability Requirements

- The system should support high uptime service for core services.
- For complex operations (e.g., submitting complaints with attachments, adding comments with notifications), database transactions should be used to ensure data integrity and to roll back the transaction if a failure occurs.
- It should be configured to have retry and optimistic locks in the e-mail queue processor to prevent: double sends and to gracefully deal with SMTP failures.

3.3.5 Scalability Requirements

- It should be scalable and accommodate growing number of users and complaints, while maintaining performance.
- The well normalized database schema (17+ tables) needs to be able to accommodate data growth efficiently.
- The structure should be modular for new features to be added.

3.3.6 Maintainability Requirements

- The code should be very structured, logic (includes/functions.php), UI (includes/ui.php), and database (includes/db.php).
- They are included in the header, sidebar, and in certain modules.
- The setup must be centralized (like database.php, mail_config.php) and
- environment-based (.env).

- Comprehensive validation functions should be used in the same way throughout the application.

3.4 Overall Description

3.4.1 User Needs

The system is a web application that will allow for one place within MUST to submit, track Handle student grievances and complaints. It simplifies the complaint handling process, without having to go through the hassle of The necessity to resort to manual forms, email and phone calls. The key roles in the three are as follows: The system: Student, Staff, Administrator. It is a forum where students can submit a complaint, keep track of complaints, and take follow-up action. make comments and withdraw complaints as appropriate, and work towards progress. Staff can handle assigned complaints, update on the status, prioritize the work, include internal notes and forward to Other staff for co-ordination. Full access to system control, user management, routing. The admin has unlimited number of configurations, role assignments and analytics. This solution will improve communication, accountability and efficiency of the processing of All complaints about the University in one application.

3.4.2 Product Perspective

The UniResolve system is designed to be a stand-alone, self contained Web application. It works independently and is accessible via the normal web browser. Provides end-to-end solution. as well as its own asynchronous notification queue via email for the entire complaint life cycle, It is not required to rely on third-party message brokers as complex as this.

3.4.3 Product Function

Identifying the following product functions is possible based on the project's goals and objectives:

- User Authentication & Profile: Secure login/logout, password management and student profile updates.

- Complaint Submission: Students have the ability to submit individual/collective complaints with attachments.
- Auto-Routing: System automatically routes complaints to the right staff, department and type.
- Status Management: Change status (Pending → In Progress → Resolved/Rejected).
- Communication: Both two-way commenting and internal staff notes are private.
- Complaint Forwarding & Priority: Staff can forward complaints and give priority (Normal, Medium, High).
- Notifications: In-app notification center and async email notifications.
- Assessment: Standard, customized, and student-specific tracking of student assessments.
- System overview, KPIs, department performance, and activity logs – Admin Dashboard & Reports.
- RBAC & User Management: Admin control of users, roles, departments, types and routing rules.

3.4.4 Operating Environment

The following tools are used to develop the proposed system:

Software Requirements:

Operating System: Windows 11 (Development), Linux (Production)

Web Server: Apache (XAMPP for Development)

- Development Language: PHP 8.2
- Languages: JavaScript, PHP, MySQL, Java, C++, Python, and SQL Server
- Database: MySQL / MariaDB 10.4

The PHPMailer (for SMTP emails) library is present.The PHPMailer library for SMTP emails is available.

We will be using Visual Studio Code as our IDE.

- Browser: Google Chrome / Microsoft Edge

Hardware Requirements:

- Processor: Intel Core i5
- Hard Disk: 256 GB SSD
- Memory: 8 GB RAM

3.5 Process Model

The Spiral Model has been chosen as the software development process model to be used for the development of UniResolve. The Spiral Model is suitable for projects that are iterative, in which requirements are continually changing throughout the project and in which the users' feedback is critical to completing system functionality.

3.5.1 Why use the Spiral Model

There are various complicated modules to be implemented in the proposed system including Authentication, Dynamic complaint submission (collective vs individual), Routing, RBAC, Asynchronous email queues, and Admin analytics. The Spiral Model can be very effectively used as a combination of Design in stages and Prototyping in stages and facilitates:

- Gradual implementation of modules (e.g., first core complaint process, then notifications, then RBAC, and then collective complaints).
- Identification and mitigation of risk each step of the way.
- Ongoing improvements to reflect user and stakeholder feedback.
- The ability to adapt to functional or user needs.

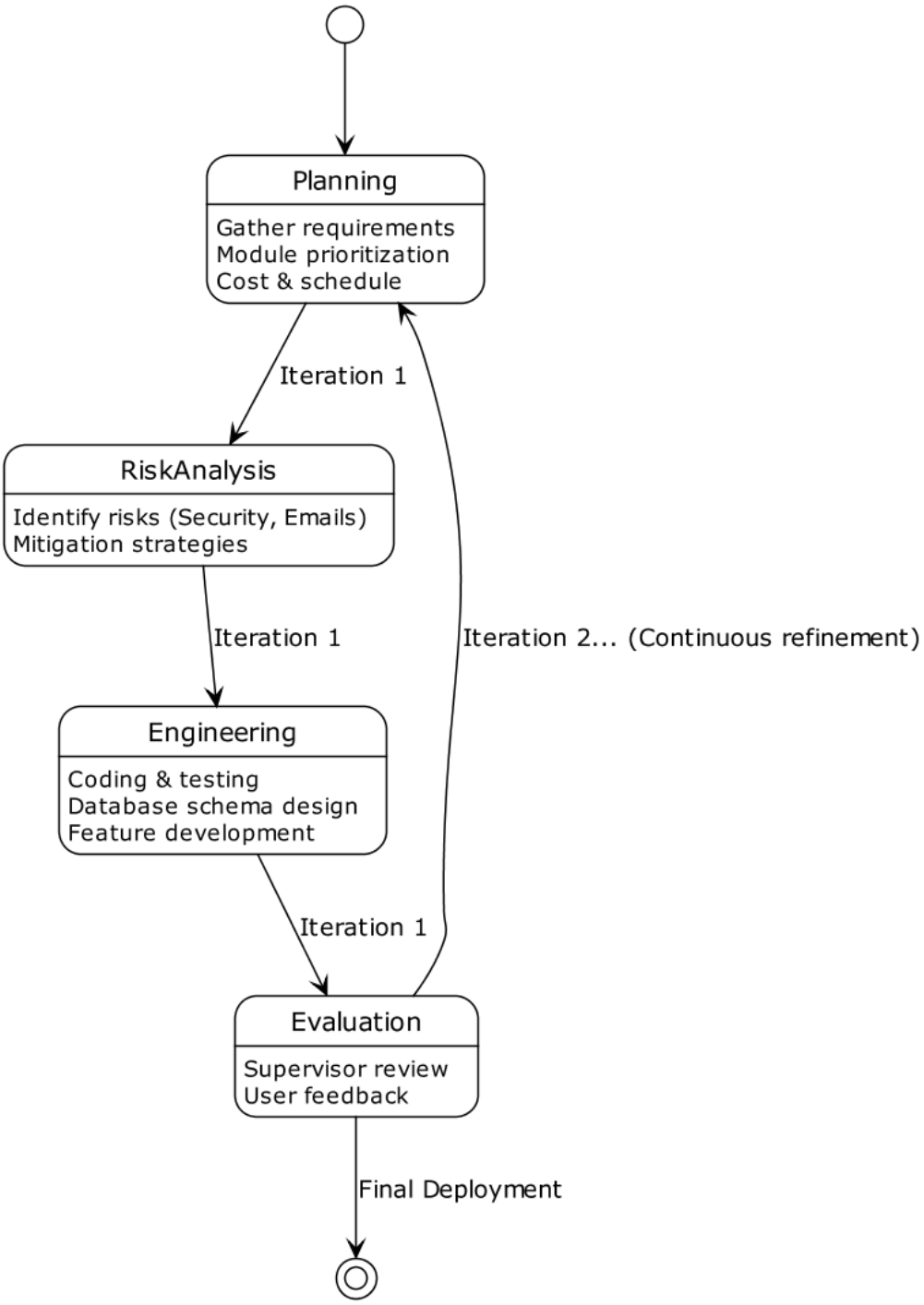


Figure 3.1: Spiral Model for UniResolve development

3.5.2 Spiral Model Phases

Planning Phase

- Requirements collecting and examination for grievances control process.
- Identify module priorities for authentication, submission, routing, notifications/comments, admin tools.
- Estimating costs and scheduling.

Risk Analysis

- Potential risks identified such as data security, email delivery issues and user adoption.
- Planned risk mitigation (e.g. async email queue so UI is not blocked).

Engineering

- Iterates coding, testing, and deployment of the system modules.
- Schema design and normalization of the database, and incremental changes (such as adding tables for comments and notifications).

Evaluation

- Supervisor and test user evaluation of the system.
- Defining and tracking risk including scope creep and change of requirements.

3.6 Summary

In this chapter, we gave a detailed review of the requirements for UniResolve. The functional requirements added additional features to be expected from the system, including user authentication, complaint submission (collective complaints, withdrawal of complaints), auto-routing, status management, communication (comments, internal notes), notifications and thorough admin capabilities. These were supported by the non-functional requirements associated with the performance (async processing), security, usability, scalability and maintainability.

The systems purpose, operating environment and multiple user roles were described at a high level in the overall description. The Spiral Model was chosen as a development method because it has an iterative, feedback loop process that closely follows the nature and complexity of this project and its modular design. This requirement analysis will provide a strong base line for the design, development and implementation of the system.

Chapter 4

Feasibility Study

4.1 Feasibility Study

A feasibility study is carried out to evaluate the proposed system from technical, economic, operational and legal points of view and determine whether it is practical, viable and beneficial. It assists in pin-pointing possible limitations and guarantees that the software program is created and delivered successfully within the constraints.

A centralized platform to handle complaints for MUST (UniResolve) has been assessed along the following dimensions:

- Technical feasibility study
- Economic feasibility study
- Operational feasibility study
- Legal feasibility

4.2 Technical Feasibility

Technological feasibility of the application – are there technologies available today to build this application?Technological Feasibility – is the application technically feasible using modern web development technologies? It is written in PHP 8.2 on the server, MySQL/MariaDB for database management and tailwind for a responsive front-end design. It is also backed by PHPMailer with a good and reliable SMTP email system.

- The technologies (PHP, MySQL, Tailwind CSS) required are widely available, well-documented and open-source.
- There are no restrictions on access to development tools (VS Code, XAMPP).
- Modular system architecture for scalable and maintainable development.
- PDO prepared statements and custom validation functions ensure safe database interaction.
- The asynchronous email queue processor (process_queue.php) allows that delivery of email is not blocking the user interface, and exhibits a high level of technical feasibility with regard to performance optimization.
- Secure deployment with environment based configuration (.env).

4.3 Economic Feasibility

The development cost will be very low since all the technology stack is open-source and free. PHP, MySQL, Tailwind CSS, PHPMailer and XAMPP are all free of licensing fees. The development and testing tools (VS Code, web browsers) are freely available.

- No licensing fees – all technologies are open source.
- Low cost development environment – runs on standard hardware.
- Support for all standard LAMP (Linux, Apache, MySQL, PHP) or WAMP (Windows, Apache, MySQL, PHP) compatible web servers – XAMPP or standard server package.
- Unlike paid options, such as Zendesk and Freshdesk, there are no recurring subscription charges.
- Improved administrative efficiencies and opportunities for cost savings due to elimination of manual complaints process.

4.4 Operational Feasibility

Overall, the application has proven to be beneficial for its users as it provides a centralized, transparent, and efficient complaint management system. It is meant to be accessible, easy to use and simple for a wide range of people, even for those who are not very technical.

- Implements a streamlined process for complaints, communication and accountability for the University.
- Role based dashboards (Student, Staff, Admin) provide tailored access and functionality for each user type.
- Responsive design = Designed to be easily used from desktop, tablet, and mobile devices.
- Saves manual effort with automation features such as automated routing, prioritized task lists and bulk import tools (CSV).
- Reduces the need of using paper forms, misplaced email, and phone calls, consolidating communication in the app through comments and notifications.

4.5 Legal Feasibility

All the tools and technologies employed in this project are open source and legally accessible to academic and institutional purposes.

- PHP is licensed under PHP License, allowing freely use and distribution.
- MySQL/MariaDB comes with GPL (General Public License).
- Tailwind CSS is available under the MIT License.
- The PHPMailer software is licensed under the LGPL (Lesser General Public License).
- XAMPP is a free package for development purposes.
- It provides secure storage of user data by using password hashing (bcrypt) and prepared statements – common data protection mechanisms.

4.6 Tools and Technologies

These are the tools and technologies used for this project:

4.6.1 PHP 8.2

PHP (Hypertext Preprocessor) is an open-source server-side scripting language used for web development that is widely used. It's used inside HTML and is well suited to developing dynamic Web pages and Web applications.

The main back-end language used by UniResolve is PHP, which is used for all backend logic such as authentication, database access, form validation, complex complaint routing logic, email queue processing (asynchronous) and session handling.

Some of the important features of PHP used were:

- Server-Side Processing: Runs on the server and returns dynamic HTML.
- Seamless database integration using PDO (PHP Data Objects) and MySQL/MariaDB.
- Session Management: Session feature to store and manage user state and security between requests.
- Security Functions: Native password hashing (bcrypt).
- File Handling: Securely managing the handling of file attachments with complaint.

4.6.2 MySQL / MariaDB

MySQL is an open source relational database management system (RDBMS) that uses Structured Query Language (SQL) to manage the data. MariaDB is a fork of MySQL that is

developed by a community that is fully compatible with MySQL but provides better performance.

MariaDB is the main database used for UniResolve and contains all data on 17 normalized tables such as User Accounts, Complaints, Routing Rules, Notifications, Comments and Priority Histories. The database is transaction supported with InnoDB storage engine.

The following are some of the most important features used:

- Relational Data Model: Well-structured tables with clear relationships (e.g., complaint_students for collective complaints).
- Transaction Support: Commit/rollback mechanism that is widely used to guarantee data integrity in multi-table inserts such as complaint submission, comment posting.
- Optimistic Locking: This is used in the email queue system to ensure that only one processor can send the same email.

4.6.3 Tailwind CSS v4

Tailwind CSS is a CSS framework designed to be a utility-first framework, with low-level utility classes that can be used to create custom user interfaces within HTML versus CSS.

The entire frontend design for UniResolve, ranging from responsive layouts and role-based navigation, dashboard cards and data tables, dynamic modal forms, timeline visualizers, and status badges, are built using Tailwind CSS.

The following key features have been used are:

- Utility-First Approach: Pre-built utility classes for quick UI development.
- Responsive Design: In-built responsive breakpoints for mobile, tablet and desktop layouts.
- Modern UI Elements: Shadows, flexbox/grid layouts, transitions, and modern UI design patterns.

4.6.4 PHPMailer

A powerful, well-documented and fully functional class for building and sending email using PHP.

It is used in UniResolve to send secure (STARTTLS) SMTP mail when someone complains, notifies an update or sends a notification.

4.6.5 XAMPP & Visual Studio Code

XAMPP includes all the local development and testing environment (Apache web server and MariaDB database).

For writing PHP, HTML, CSS and JavaScript code, Visual Studio Code (VS Code) was the IDE of choice with the help of extensions for the Tailwind CSS IntelliSense.

4.6.6 UI/UX Design

The concepts of UI/UX (User Interface/User Experience) design principles were crucial in developing an intuitive and engaging web interface for UniResolve.

The key aspects of UI/UX design that have been put in place are:

- The concepts of UI/UX (User Interface/User Experience) design principles were crucial in developing an intuitive and engaging web interface for UniResolve.
- The key aspects of UI/UX design that have been put in place are:
- User-Centered Design with role specific dashboards, a dedicated notification center and interactive complaint tracking timelines.
- Responsive Design – making sure that the interface looks good on both desktop and mobile devices (e.g., collapsible sidebars).
- Consistent typography, colors (as Pending, In Progress, Resolved, Withdrawn statuses) The guides for the old and the new card based layouts.
- colour coding, the user can easily identify what is going on.- clear labelling, colour coded status indicators and intuitive pattern of use make it easy for the user to know what's happening. navigation.

4.7 Summary

The chapter thoroughly evaluated the feasibility from several angles of developing UniResolve. The technical feasibility report was given and certified that the Using common open source tools and technologies, project can be developed. which are PHP, MySQL,

Tailwind CSS and PHPMailer. The advanced architectural Uniresolve solutions, configuration via the `.env` file and an asynchronous queue for solutions were implemented. Underlines the technical foundations of the system, in emails. Economic viability proved the system is viable if implemented at no license fee. Which is perfect for academic use. Its feasibility with respect to its usage has been proven. and demonstrated, and the system is proved to be very useful at universities with regards to centralisation. Complaint management, communication tools and automated workflows. The legal feasibility ensured that all tools and technologies used are open source and readily available. Overall the feasibility study concludes that UniResolve is viable, economical, technically and operationally ready. Easily applicable and very useful for the intended target group. This positive evaluation assists in designing and development of the application.

Chapter 5

System Design

5.1 Introduction

The system design of UniResolve is described in detail in this chapter through diagrams, which show the architecture and processes of the complaint management system. The system design phase is an essential step in the system development process, where the conceptualized system starts to materialize. This phase is about turning the requirements collected in the analysis phase into a detailed design of the system, including its architecture, components, and functions.

This chapter shows the Use Case Diagrams, Use Case Description Tables, Data Flow Diagrams (DFDs) and Entity Relationship Diagram (ERD) of the system, making it well planned for development, integration and deployment.

5.2 Use Case Diagrams

Use Case diagram is used to show the relationship between the user and system. Use Cases represent functions of the system. It is used directly or indirectly by users. Can also be called a behavior diagram.

5.2.1 Student Use Case

The Student role use case diagram. A student logs on to the system with his/her University username and password. After identifying as a student, the student may file new individual or collective complaints by choosing a category, submitting a title and description, and, if desired, also including files. The student may use a unique tracking ID to track current complaints, access their complete complaint history, withdraw complaints pending, post comments to staff, manage their profile and see in-app notifications.

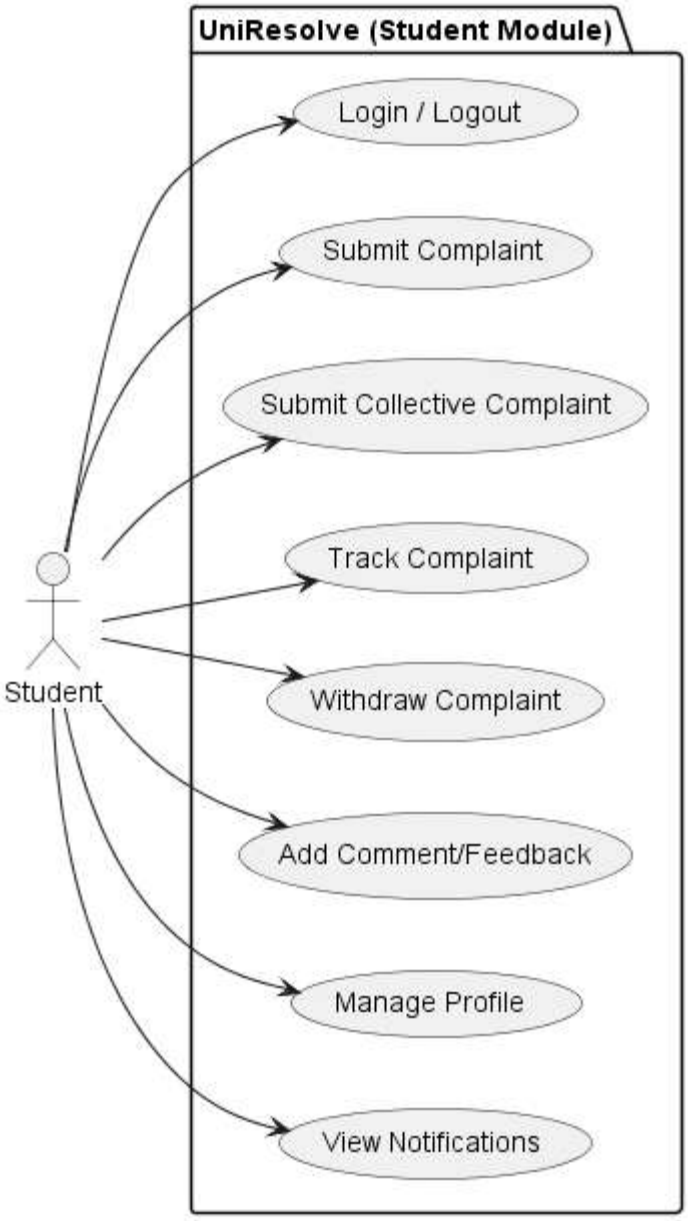


Figure 5.1: Use Case Diagram for Student

5.2.2 Staff Use Case

The Staff use case diagram illustrates the staff's capabilities. Once logged in, staff will be able to see their designated complaints dashboard, change complaints status, add private internal notes, reply to student notes, forward complaints to other staff, change priority and see notifications. CMS Coordinators also can use bulk student import (CSV) and role assignment tools.

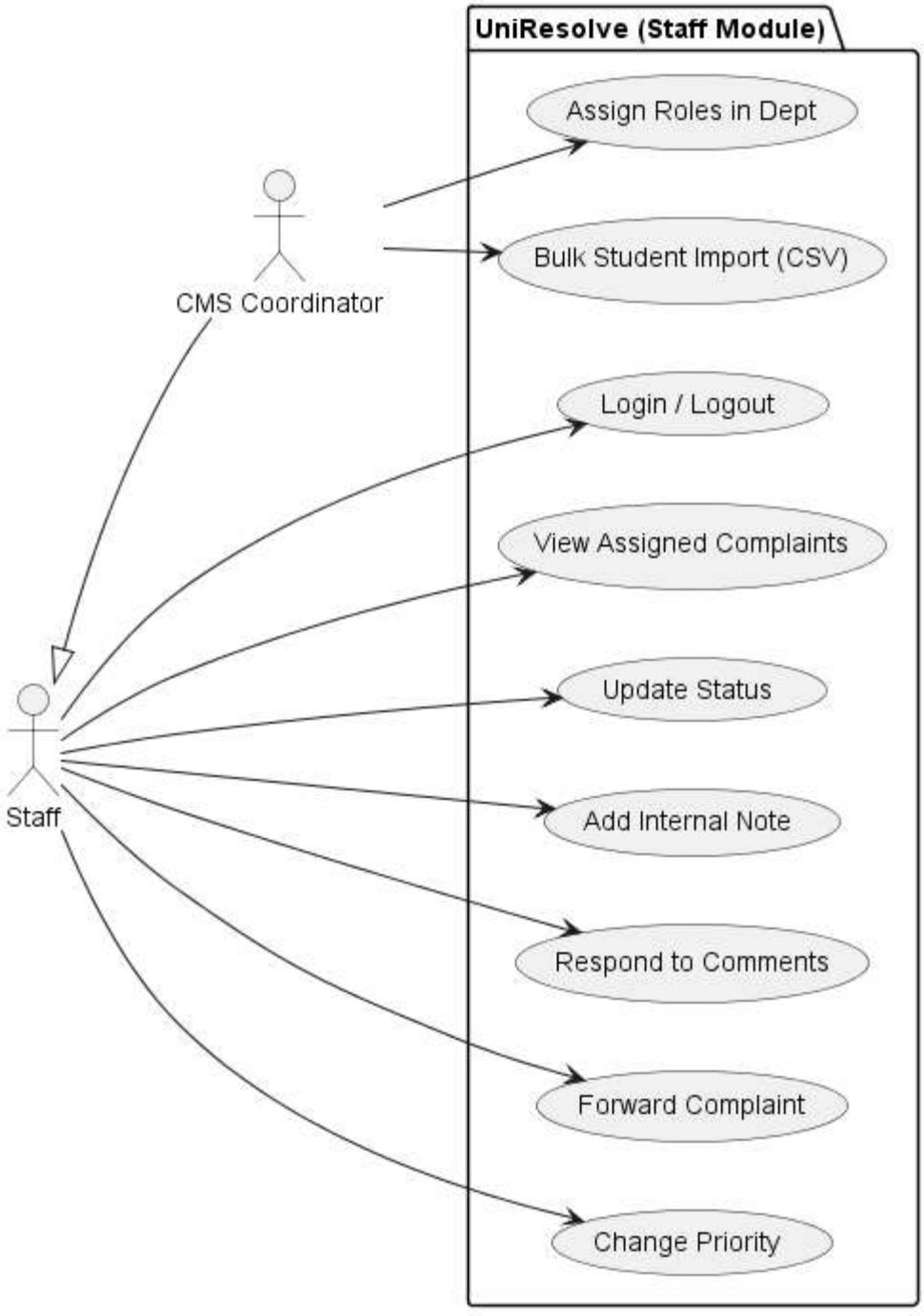


Figure 5.2: Use Case Diagram for Staff

5.2.3 Admin Use Case

Use case diagram of Admin. The admin has complete system control. Once signed on, the admin can manage users (create, activate, deactivate), manage roles, manage departments,

manage complaint types, configure auto-routing rules, view all complaints in the system, monitor system activity logs and generate in-depth reports and analytics.

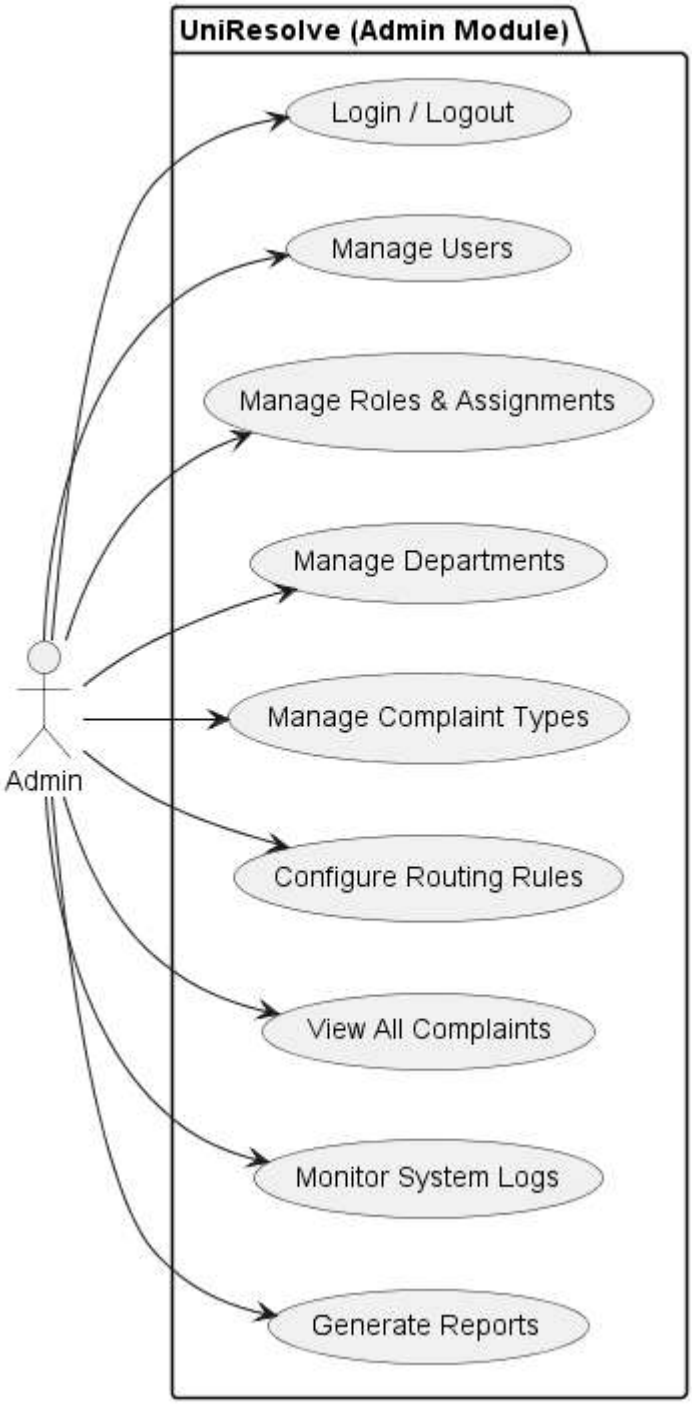


Figure 5.3: Use Case Diagram for Admin

5.3 Use Case Description

The tables explaining the use cases are as follows:

5.3.1 Use Case Table for Login

Field	Description
Use Case ID	UC-01
Use Case Name	Login to System
Brief Description	Allows registered users to authenticate and access their role-specific dashboards.
Actors	Student, Staff, Admin
Pre-conditions	User must have a registered, active account in the system.
Post-conditions	User session is established and user gains access to the system dashboard.
Triggering Event	User navigates to the portal and attempts to access secured features.
Basic Flow	1. User enters Login ID (Email) and Password. 2. User clicks the "Login" button. 3. System validates credentials. 4. System redirects user to the appropriate role-based dashboard.
Alternative Flows	None
Exception Flows	3a. Invalid credentials: The system displays an error message ("Invalid email or password"). 3b. Inactive Account: The system denies access and displays a deactivation notice. 3c. Password Change Required flag is true: System redirects user to the Change Password screen instead of the dashboard.
Included Use Cases	None

Table 5.1: Use Case Table for Login

5.3.2 Use Case Table for Submit Complaint

Field	Description
Use Case ID	UC-02
Use Case Name	Submit Complaint
Brief Description	Enables students to file a new individual or collective complaint, including file attachments.
Actors	Student
Pre-conditions	Student is authenticated and logged into the student dashboard.
Post-conditions	Complaint record is securely saved in the database with "Pending" status and assigned to appropriate staff.
Triggering Event	Student clicks the "Submit Complaint" navigation button.
Basic Flow	1. Student selects the complaint category and specifies if it is "Individual" or "Collective". 2. Student enters a title and a detailed description. 3. System generates a unique tracking ID. 4. System queries routing rules and auto-assigns the complaint to a staff member. 5. System dispatches a confirmation email to the student and a notification to the assigned staff.
Alternative Flows	1a. Student joins an existing Collective Complaint: Instead of creating a new record, the student is added to the complaint_students mapping table.
Exception Flows	2a. Missing required fields: System highlights empty fields and blocks submission. 2b. Invalid file attachment (e.g., EXE file): System rejects the upload and displays an error message.
Included Use Cases	None

Table 5.2: Use Case Table for Submit Complaint

5.3.3 Use Case Table for Add Comment

Field	Description
Use Case ID	UC-03
Use Case Name	Add Comment / Feedback
Brief Description	Allows bidirectional communication between the student and the assigned staff on a specific complaint.
Actors	Student, Staff
Pre-conditions	User is logged in. The specific complaint exists and its status is not "Withdrawn".
Post-conditions	The comment text is saved to the complaint thread, and the recipient is immediately notified.
Triggering Event	User opens the specific complaint details page and wishes to communicate.
Basic Flow	1. User types text into the "Add Comment" textarea. 2. User clicks the "Submit Comment" button. 3. System validates the text and saves it to the complaint_comments table. 4. System generates an in-app notification for the opposing party (student or assigned staff).
Alternative Flows	None

Exception Flows	3a. Comment text is completely empty: System disables submission or shows an HTML validation error.
Included Use Cases	None

Table 5.3: Use Case Table for Add Comment

5.3.4 Use Case Table for Withdraw Complaint

Field	**Description**
Use Case ID	UC-04
Use Case Name	Withdraw Complaint
Brief Description	Permits a student to cancel their own submitted complaint before it is resolved.
Actors	Student
Pre-conditions	Student is logged in. Complaint status is "Pending" or "In Progress". Complaint type must be "Individual".
Post-conditions	Complaint is marked as "Withdrawn". No further action is required from the staff.
Triggering Event	Student decides they no longer need the complaint resolved.
Basic Flow	1. Student navigates to the Track Complaint page. 2. Student clicks the "Withdraw" button. 3. System prompts for final confirmation via a modal dialog. 4. Student confirms the withdrawal. 5. System updates the status to "Withdrawn" and inserts an entry into the status history table.
Alternative Flows	None
Exception Flows	1a. Complaint is already Resolved or Rejected: System hides the Withdraw button entirely.

	1b. Complaint is Collective: System disables the Withdraw function to protect the interests of other participating students.
Included Use Cases	None

Table 5.4: Use Case Table for Withdraw Complaint

5.3.5 Use Case Table for Update Status

Field	**Description**
Use Case ID	UC-05
Use Case Name	Update Complaint Status
Brief Description	Staff members can change the progression status of the complaints currently assigned to them.
Actors	Staff
Pre-conditions	Staff is logged in and the specific complaint is assigned to their staff account.
Post-conditions	Complaint status is updated, an audit trail entry is created, and the student is informed.
Triggering Event	Staff makes progress on a complaint or resolves the underlying issue.
Basic Flow	1. Staff opens the assigned complaint view page. 2. Staff selects a new status (Pending, In Progress, Resolved, or Rejected) from the dropdown. 3. Staff types optional remarks. 4. Staff clicks "Update Status". 5. System updates the database, records the change in the history table, and notifies the student.
Alternative Flows	None

Exception Flows	2a. No status selected: System displays a validation error preventing the update. 4a. Database failure: System rolls back the transaction and shows an internal error.
Included Use Cases	None

Table 5.5: Use Case Table for Update Status

5.3.6 Use Case Table for Manage Roles

Field	**Description**
Use Case ID	UC-06
Use Case Name	Manage Roles
Brief Description	Admins can create new role archetypes and dynamically assign them to staff members.
Actors	Admin
Pre-conditions	Admin is authenticated and logged into the Admin panel.
Post-conditions	Staff member's permissions are updated immediately, unlocking new UI features for them.
Triggering Event	A staff member requires elevated permissions, such as becoming a CMS Coordinator or Session Advisor.
Basic Flow	1. Admin navigates to the Role Management portal. 2. Admin selects a specific staff member from the department directory. 3. Admin assigns a predefined role to that staff member. 4. Admin clicks "Save Configuration".

Alternative Flows	3a. Role is "Session Advisor": System dynamically displays an extra dropdown for the admin to select the specific academic session (e.g., 2022-2026).
Exception Flows	3b. Duplicate role assignment: System detects the staff already holds this role and displays an error alert.

Table 5.6: Use Case Table for Manage Roles

5.5 Entity Relationship Diagram (ERD)

An Entity Relationship Diagram (ERD) is a type of flowchart that illustrates how "entities" such as people, objects, or concepts relate to each other within a system. The ERD for UniResolve shows the relationships between all major entities in the database.

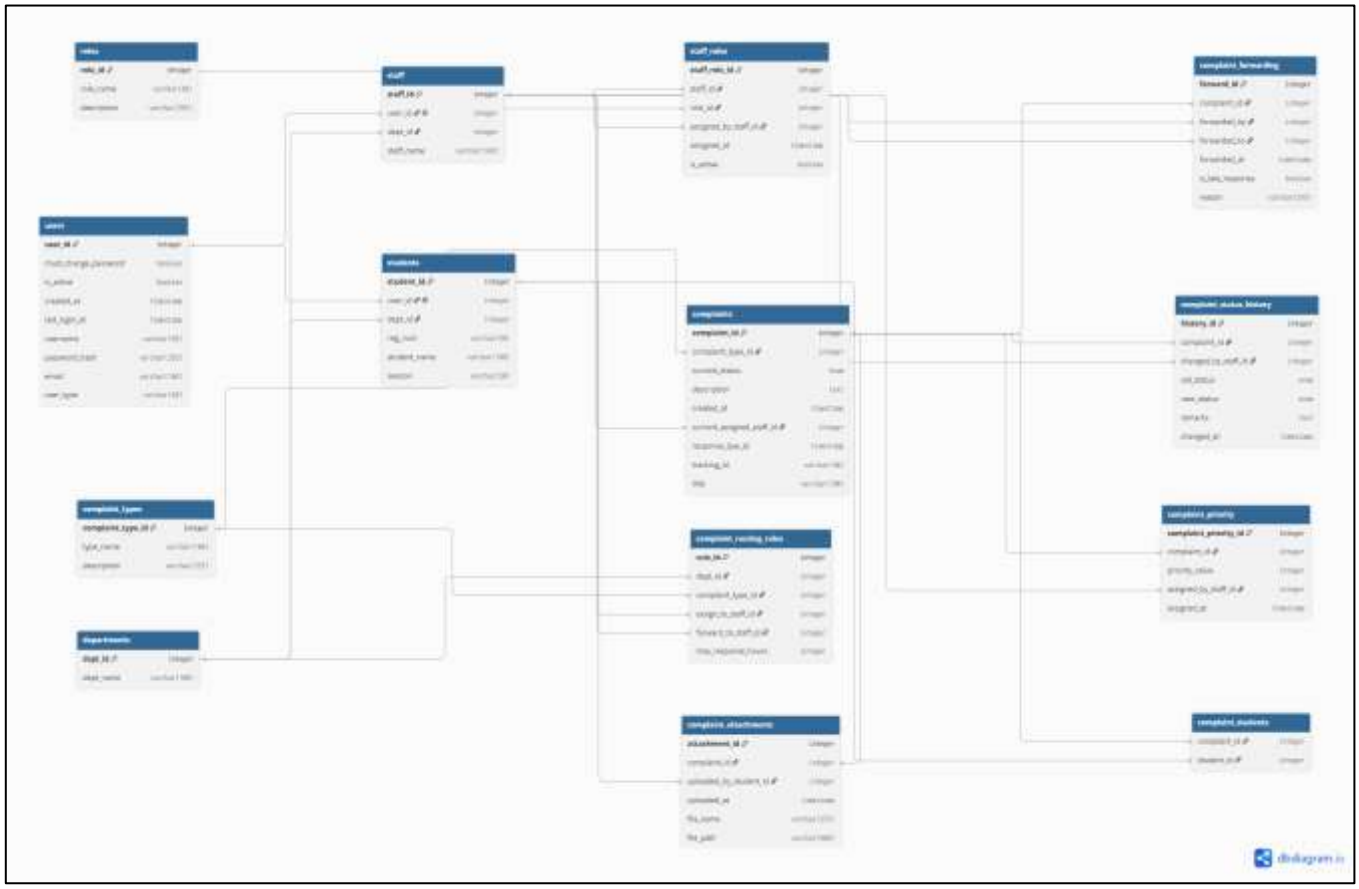


Figure 5.8: ERD Diagram of UniResolve

5.5.1 ERD Entity Descriptions

Entity	Key Attributes	Description
users	user_id, login_id, password_hash, email, user_type	Central authentication table for all users
students	student_id, user_id, dept_id, reg_num, session	Student profile linked to users table
staff	staff_id, user_id, dept_id, staff_name	Staff profile linked to users table
departments	dept_id, dept_code, dept_name	University departments with unique codes
roles	role_id, role_name	System roles (Admin, HOD, CMS Coordinator, Session Advisor)
staff_roles	staff_id, role_id, advisor_session	Many-to-many mapping supporting session advisors
complaints	complaint_id, tracking_id, current_status, priority	Core complaint records
complaint_types	type_id, type_code, type_name	Complaint categories (e.g., ACAD, ADM)
complaint_students	complaint_id, student_id	Many-to-many mapping for collective complaints
complaint_status_history	history_id, complaint_id, old_status, new_status	Audit trail of status changes
complaint_comments	comment_id, complaint_id, user_id, comment_text	Bidirectional student-staff feedback
notifications	notification_id, user_id, message, is_read	In-app notification system alerts
email_queue	email_id, recipient, subject, status	Queue for asynchronous email processing

complaint_routing_rules	rule_id, dept_id, type_id, assign_to_staff_id	Auto-routing configuration
complaint_internal_notes	note_id, complaint_id, staff_id, note_text	Private staff notes on complaints

Table 5.7: ERD Entity Descriptions for UniResolve

5.5.2 ERD Relationships

Relationship	From Entity	To Entity	Type	Description
Has Profile	users	students / staff	One-to-One	Each user has one specific profile
Belongs To	students / staff	departments	Many-to-One	Users belong to one department
Has Roles	staff	roles	Many-to-Many	Staff can have multiple roles via staff_roles
Submits	students	complaints	Many-to-Many	Students and complaints linked via complaint_students (collective)
Assigned To	complaints	staff	Many-to-One	Multiple complaints assigned to one staff
Has Type	complaints	complaint_types	Many-to-One	Complaints categorized by type
Has Comments	complaints	complaint_comments	One-to-Many	Complaints have multiple comments
Has Notifications	users	notifications	One-to-Many	Users receive multiple notifications

Has History	complaints	complaint_status_history	One-to-Many	Each complaint has multiple status entries

Table 5.8: ERD Relationships for UniResolve

5.6 Summary

The key structural and functional elements of UniResolve were introduced in this Chapter using a detailed system design diagrams and descriptions. Use Case diagrams were used to show the interactions between the three roles (Student, Staff, and Admin) of the system. The Use Case description tables clearly specified the use cases, such as the login, submission of complaints, adding comments, withdrawing complaints, and role management.

Data Flow Diagrams were drawn at context level and at Level 1, which shows how the data was to be passed around between users, system processes and data stores as an introduction of the new notification and comment systems were made. The Entity Relationship Diagram was created to document the entire schema of the database, including new tables for notifications, comments, email queue and collective mappings, and their relationships, which served as a basis for database implementation.

These visuals help to understand how the system works and how it is designed, so it can be well planned for the implementation phase.

Chapter 6

Implementation & Testing

6.1 Features and Results

The main purpose of UniResolve was to provide a clear and streamlined, centralized complaints process. management system for universities. All planned features have been successfully meet. role-based authentication, individual/combining complaints, and many others, have been put in place.role-based authentication, individual/combining complaints, and more have been put in place. automated routing, status tracking, in-app notifications, bidirectional comments, priority management, administration analytics. The next sections highlight the major interfaces and functionality of the application.

6.1.1 Login Page

The login page is the first page that all users will see. It has a neat and modern user interface which includes an e-mail/login ID field and a password area. The system verifies the user with the database and directs him/her to the student, staff or admin dashboard depending on his/her role. Users are referred to change their password before others features are available if the must_change_password flag is set.

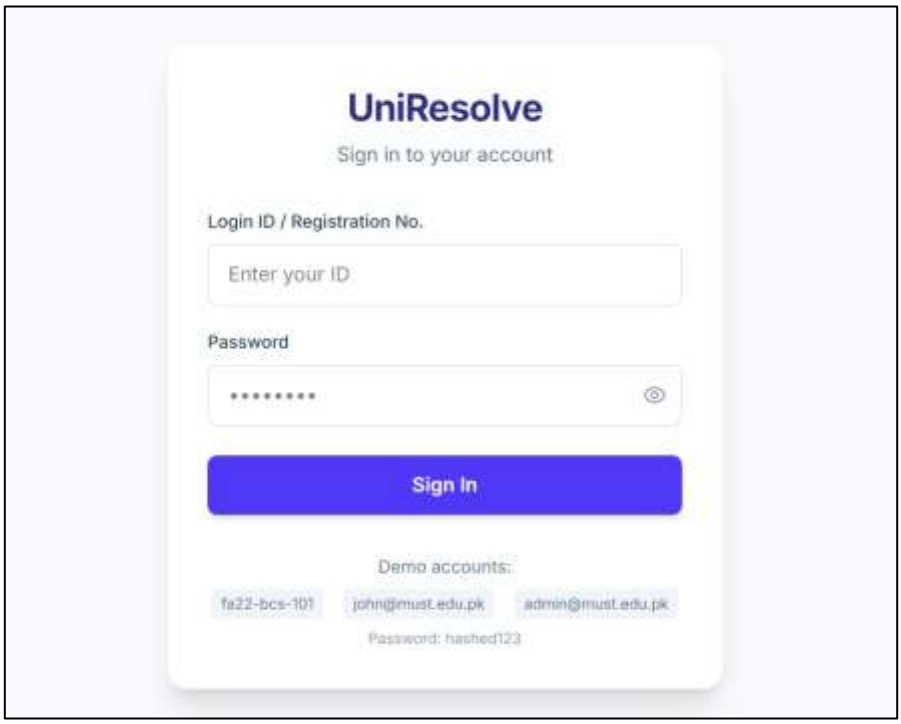


Figure 6.1: Login Page of UniResolve

6.1.2 Student Dashboard

The student dashboard offers a view of the student's complaints. It presents KPI cards with the number of Pending, In Progress and Resolved complaints. A table of student's recent complaints with their tracking ID, title, status, and submission date is provided below the cards. There is a new interactive navigation bar which features a notification bell to alert students as to real-time status updates and comments.

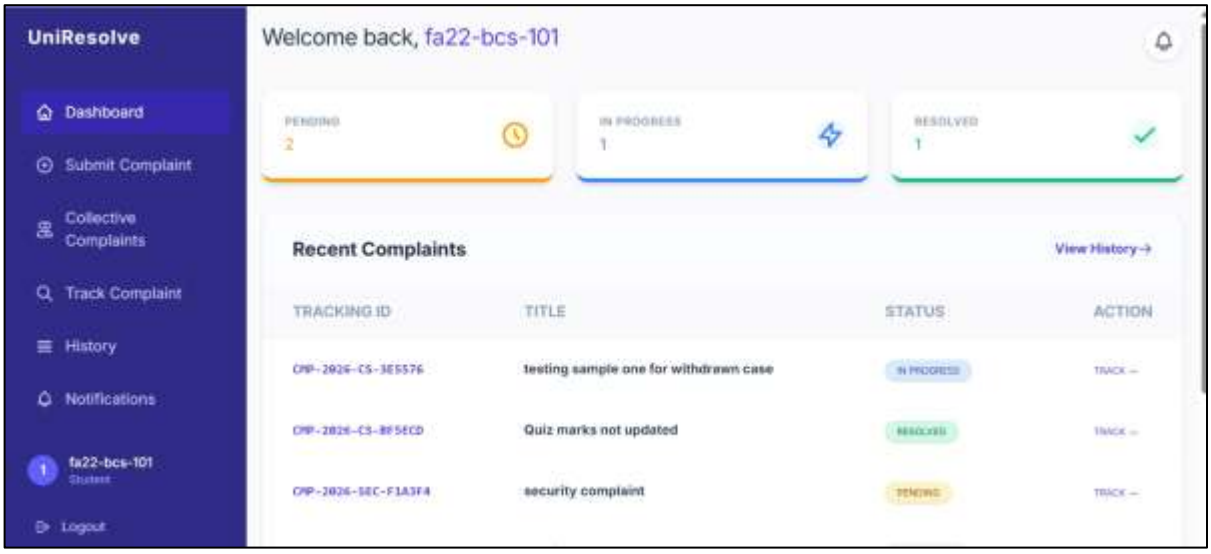


Figure 6.2: Student Dashboard

6.1.3 Submit Complaint Page

Students may choose from several options on the complaint submission form, type a complaint title and description, and optionally attach a file to support their complaint. A new feature allows students to specify if the complaint is Individual or Collective. If collective, other students in the same department will be able to view and add to the existing complaint, which will greatly decrease duplicate complaints.

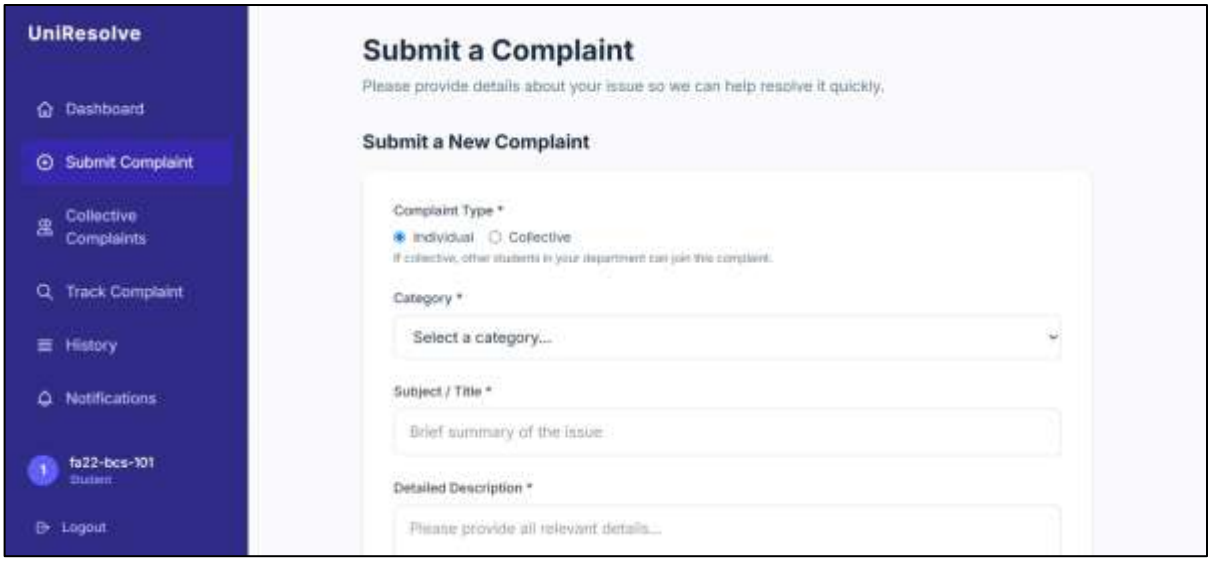


Figure 6.3: Submit Complaint Page

6.1.4 Track Complaint & Comment System

The track complaint page shows all the details of the complaint, including title, tracking ID, category, assigned staff name, complaint description and attachments. The complete status history is displayed in a visual timeline. Most importantly, it includes a new bidirectional section for comments/feedback, for students and staff to communicate directly about the complaint. It also has a 'Withdraw' option for pending complaints.



Figure 6.4: Track Complaint Page with Comments

6.1.5 Staff Dashboard & Assigned Complaints

All complaints assigned to the signed in staff member will be displayed on the Staff dashboard. It has smart sorting algorithms which sort complaints by the following priority: High priority first, then oldest. The interface makes outstanding complaints stand out in red to get attention of the staff to deal with time-sensitive complaints.

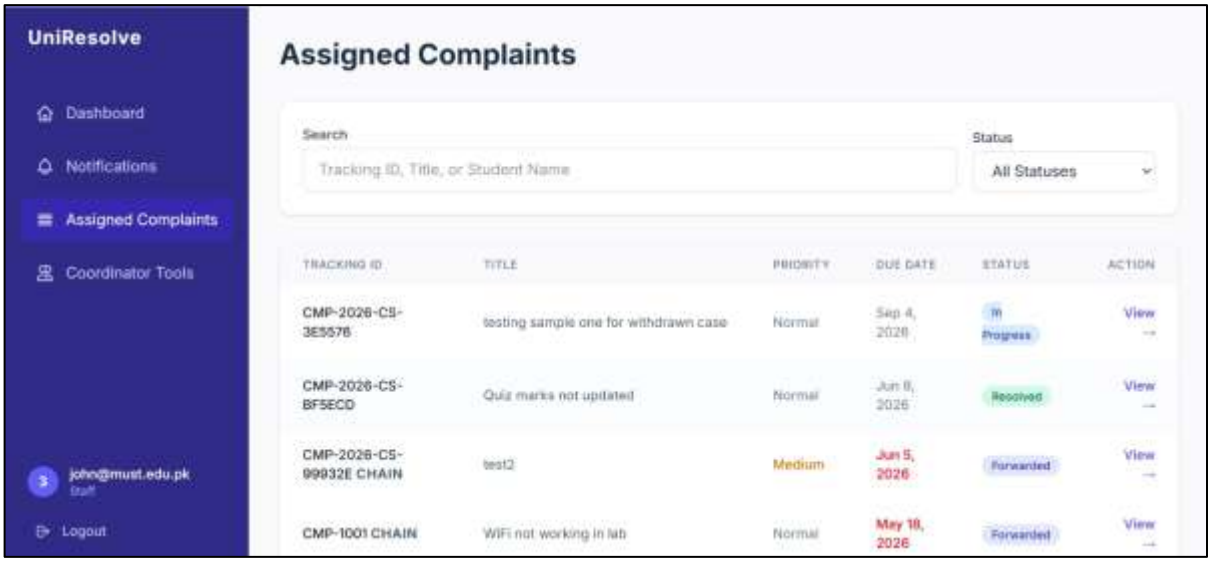


Figure 6.5: Staff Dashboard

6.1.6 Staff View Complaint Page

This is the main staff dialogue for handling a grievance. It includes action buttons to update status, forward to colleague and change priority. Staff have the ability to reply to students' comments, see the full history of a student's status and to include private notes that only show up for the staff member who gave the comment.

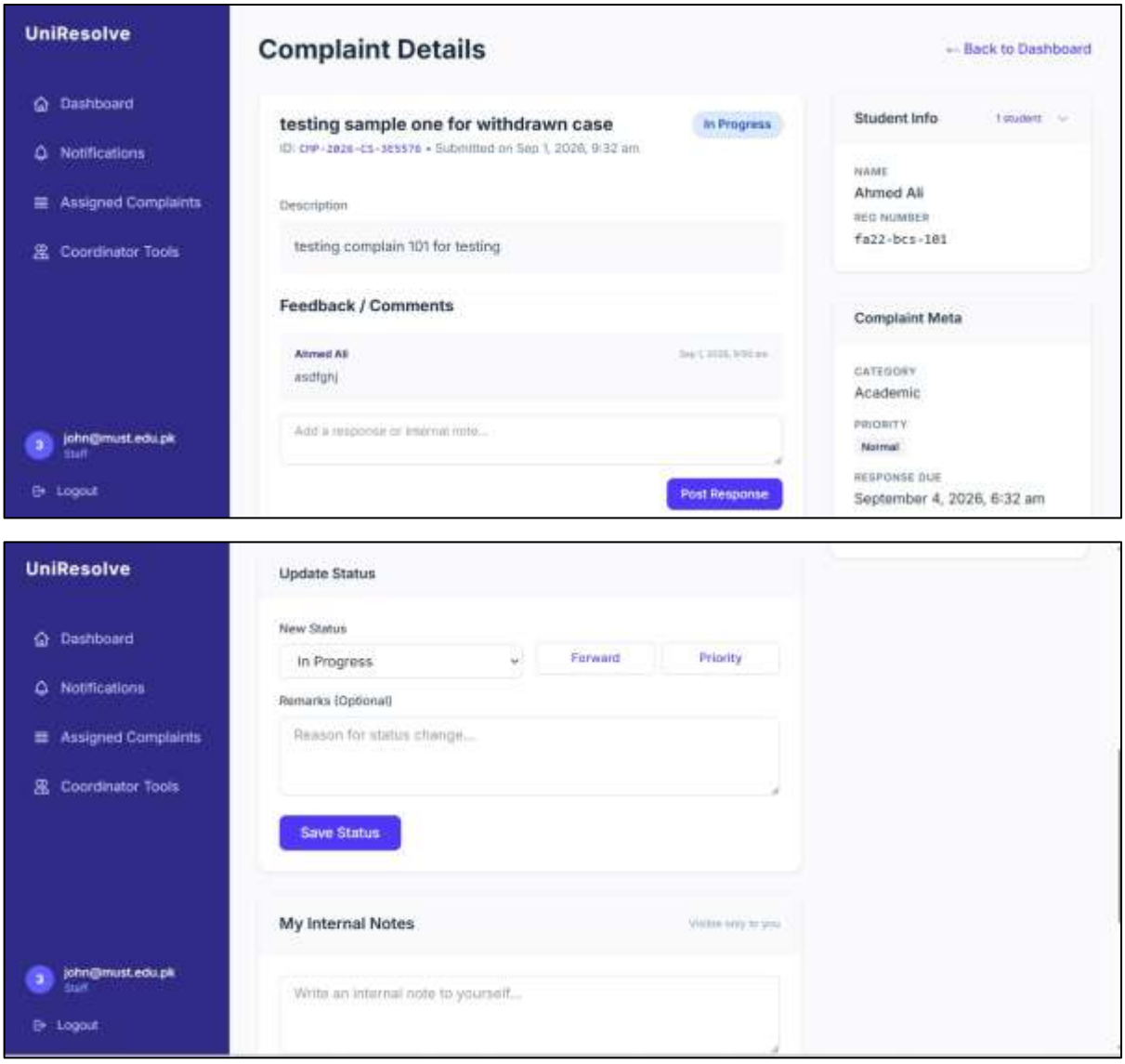


Figure 6.6: Staff View Complaint Page

6.1.7 Admin Dashboard & Analytics

The admin dashboard offers a system-wide overview with KPI cards indicating the numbers of Total, Pending, In Progress, Resolved, and Rejected complaints. A department breakdown section shows complaints made by department and has progress bars. The recent activity feed reflects the status changes and forwarding events of the entire system, in real-time.

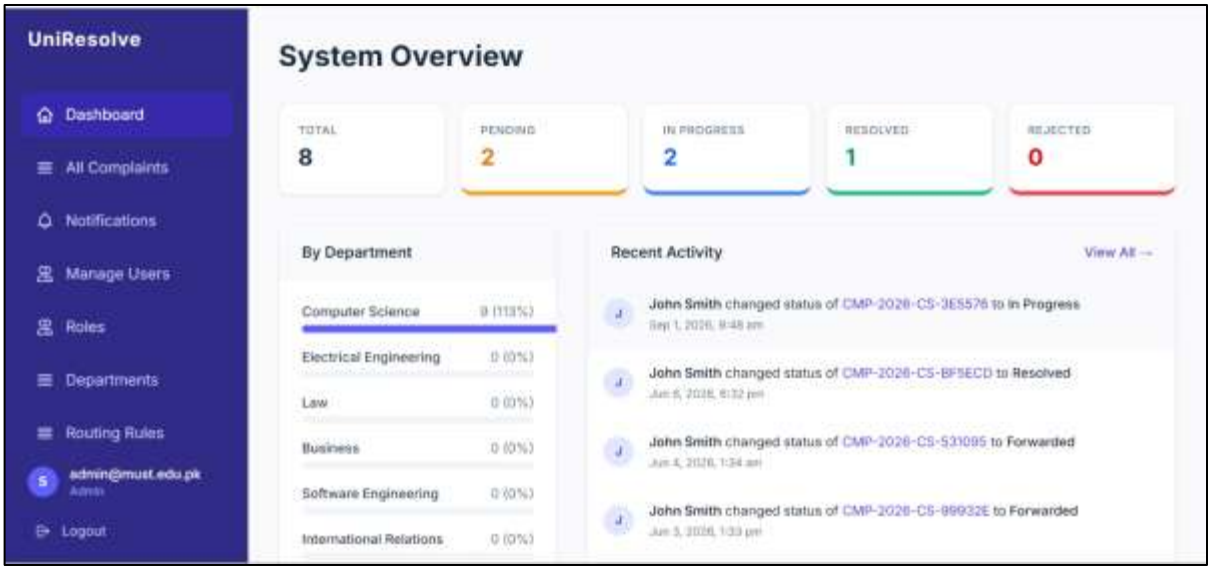


Figure 6.7: Admin Dashboard

6.1.8 Manage Roles & Users Page

These can be created on the manage roles page as special roles (for example HOD, CMS Coordinator, Session Advisor) and given dynamically to staff, based on their department.

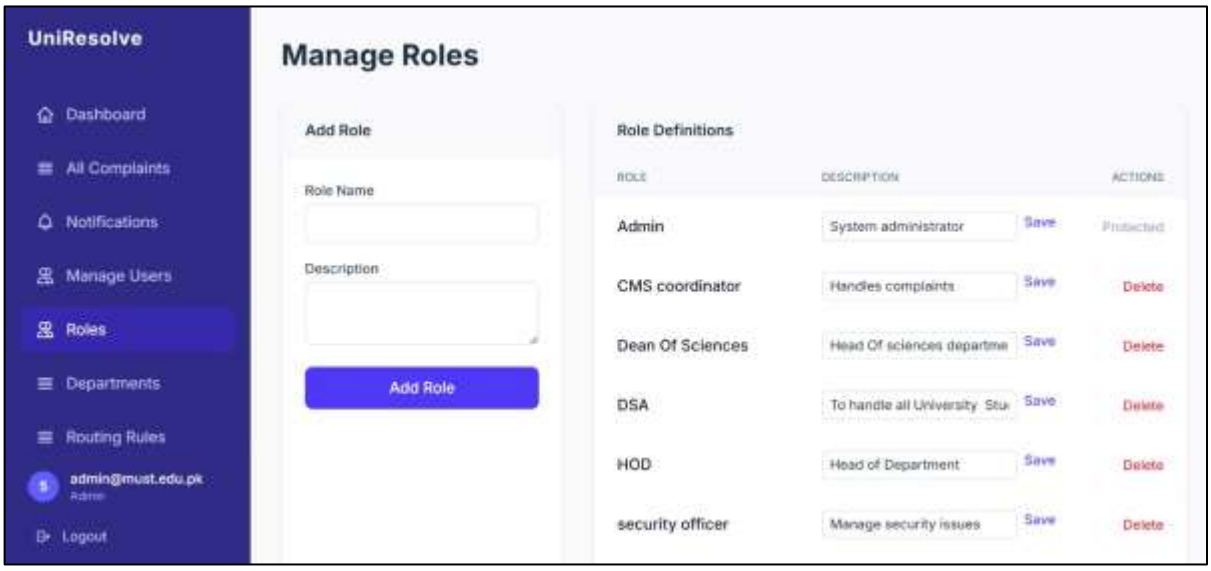


Figure 6.8: Manage Roles

6.1.9 Manage Routing Rules Page

Admins can set up automatic complaint assignment in the routing rules page. Each rule is a Department + Complaint Type combination that is assigned to a particular individual in the department, and may include forward-to-staff and max-response-time (hours). The system will be auto-assigned to a student when a complaint is submitted, based on the following rules.



Figure 6.9: Manage Routing Rules Page

6.2 Testing

Software testing is a vital step in the software development life cycle, where the software system is thoroughly examined and tested to uncover and resolve any bugs or defects. The aim of software testing is to ensure that the software works properly, is up to the requirements established by the users and is of high quality.

Different techniques and strategies are used for software testing. In our project black box testing and white box testing methodology has been used.

6.2.1 Black Box Testing

Black box testing is concerned about the functionality of the software without knowing the detail of the software code and structure. Testers develop Test Cases according to the software's specification and requirement.

6.2.2 White Box Testing

White box testing is a testing method in which the structure and the code of the software is studied. White box testing was used in UniResolve to test the correctness of SQL queries, the session management logic and the asynchronous email queue processor (process_queue.php).

6.3 Black Box Test Cases

6.3.1 Test Case 1: Login & Authentication

Case	Actual Input	Expected Output	Actual Output	Remarks
Valid User	Login: admin@must.edu.pk Password: password123	Redirect to admin dashboard	Redirected to admin dashboard successfully	Pass
Invalid User	Login: wrong@email.com Password: wrongpass	Error message displayed	Error: Invalid credentials	Pass
Inactive Account	Login: inactive@must.edu.pk Password: password123	Error: Account is inactive	Error: Account deactivated	Pass

Table: Test Case for Login

6.3.2 Test Case 2: Submit Complaint & Auto-Routing

Case	Actual Input	Expected Output	Actual Output	Remarks
Valid Individual	Type: Academic Title: Exam Issue Type: Individual	Complaint created with tracking ID, auto-assigned to staff based on rules	Complaint CMP-2026-CS-XXX created and assigned	Pass

Collective Join	Click "Join" on existing collective complaint	Student added to complaint_students mapping	Student successfully joined collective complaint	Pass
Missing Fields	Type: (empty) Title: (empty) Description: (empty)	Error: Required fields missing	Error message displayed	Pass

Table: Test Case for Submit Complaint

6.3.3 Test Case 3: Notifications & Comments

Case	Actual Input	Expected Output	Actual Output	Remarks
Add Comment	Student posts: "Any update?"	Comment saved, staff receives in-app notification	Comment displayed, staff notified successfully	Pass
Mark as Read	Click "Mark all as read" in notification center	All notifications set to is_read=1	Notifications marked as read (badge clears)	Pass

Table: Test Case for Notifications & Comments

6.3.4 Test Case 4: Withdraw Complaint

Case	Actual Input	Expected Output	Actual Output	Remarks
Valid Withdraw	Click Withdraw on Pending complaint	Status changed to Withdrawn, staff notified	Status updated successfully	Pass
Invalid Withdraw	Attempt to withdraw	Action denied	Error: Cannot withdraw	Pass

	Resolved complaint		resolved complaint	

Table: Test Case for Withdraw Complaint

6.3.5 Test Case 5: Update Complaint Status

Case	Actual Input	Expected Output	Actual Output	Remarks
Valid Update	Status: Resolved Remarks: Issue fixed	Status updated, history recorded, student notified	Status updated, notification created	Pass
No Status Selected	Status: (none) Remarks: (empty)	Error: Please select a status	Error message displayed	Pass

Table: Test Case for Update Status

6.3.6 Test Case 6: Role Management (Admin)

Case	Actual Input	Expected Output	Actual Output	Remarks
Assign Role	Assign CMS Coordinator to Staff X	Staff X gains Coordinator privileges	Role assigned, new menu items appear for Staff X	Pass
Invalid Role	Attempt to create duplicate role name	Error: Role already exists	Error message displayed	Pass

Table: Test Case for Role Management

Chapter 7

Conclusion and Future Enhancement

7.1 Introduction

The need for having a streamlined, transparent and centralized complaint was discovered. The need for a management system in University contexts resulted in the development of UniResolve. The main goal It was the purpose of this project to develop a formal system to receive and follow complaints of money-lenders, as well as analytics, that will take the place of handling complaints manually (physically) Any form of communication (forms, email, phone calls).

7.2 Results and Discussion

The UniResolve system has been developed and tested successfully with the following: achievements:

- Users and roles: Role Based Access Control – Implemented a successful, dynamic RBAC system for Users and roles. The core (Welfare Officer, Specialised Student roles – HOD, CMS Coordinator, Session Advisor) and the Core Support roles (HOD, CMS Coordinator, Session Advisor) are equally important. The roles of Student, Staff and Admin.
- Advanced Complaint Workflows: Students may submit complaints themselves or organize groups of students to file complaints. and be part of collective complaints. Full file attachments, withdrawal capabilities and automatic deadline calculation are supported.
- Automated Routing Engine: Admin configurable routing rules automatically route Complaints made to the appropriate staff, department and type of complaint, made in significant numbers. reducing any manual assignment delays.
- Bidirectional Communication: Developed a means to directly receive feedback from the student. develop a good application/comment culture by allowing staff and vice versa to interact with each other within the application, feedback mechanism.

- Notification Architecture: Designed a two tiered notification approach to incorporate real-time in Notification Design. To prevent app notifications and an asynchronous email queue processor with PHPMailer are available. Email blocking problems with UI.
- Tracking is transparent: Each page gets a unique tracking ID (CMP-XXXXXX) to track it. The public standalone tracking page allows checking the status without a need of login. authentication. statements, and entitles that can be revoked or deleted.
- Security Implementation: The system employs bcrypt password hashing, PDO prepared statements, and entitles that can be revoked or deleted. configuration management with regard to environment, statements to prevent SQL injection. (.env), and sanitizes all input for the prevention of XSS attacks.
- UI Design: Designed with tailwind CSS v4, it is a modern and user-friendly design. An intuitive desktop and mobile experience, with colour coded status Badges, interactive timelines and user-friendly navigational features.

7.3 Future Enhancements

Future Improvement to the UniResolve

The UniResolve is expected to be improved in the future:

- Now students and staff can chat in real-time with one another, so you can integrate They can communicate directly with each other using real-time, live chat through WebSockets for live communication and interaction. complaint resolution.
- Use advanced Analytics Dashboard to visualize charts and graphs. trends in complaints, resolution times and department performance over time with any of Two charting libraries will be used, one is Chart.js one is D3.js. Manage mobile applications: Develop & build native Android and iOS applications. This is a complaint submission/tracking system for on-the-go that will be based on the current PHP backend as a RESTful system. API.
- AI-based Complaint Classification; Classify complaints based on keywords using machine learning algorithms automatically; classify and prioritize complaints by their natural language description to provide more accurate routing.
- Automated Escalation Rules: Set up escalation rules to automatically escalate complaints Or notify the senior admin if the complaint is not addressed by its deadline.

Use the Student Feedback & Rating System to give students a chance to rate the quality of the resolution (1-5 stars) after the complain is resolved; offering measurable feedback to staff.

- Multi-Language Support: Provide Urdu and other regional languages support. Enhancing access and outreach to all at the university.
- Export and PDF Reports: Allows admin to export detailed system reports in PDF & Excel Offline formats for analysis and institutional record-keeping.

7.4 Summary

The future development and final results of the UniResolve project have been shared in this chapter. The overall aim of the digitization of the complaint management process is to: effectively completed, and with the development of clear complaints tracking, automating complaint routing, and reports/analytics for data-driven decision making.

The next challenge for the development of UniResolve, however, has shown that it is possible to use common, open-source software (such as Maple) to develop a similar product.

A web application can be an effective solution for real-world problems, thanks to the combination of PHP, MySQL, and Tailwind CSS.

university administration. The system's modular design, along with other features such as the Its scalability, robustness, and guarantees its dynamic RBAC and asynchronous email queue. maintainability.

The project has been a great learning curve in all aspects of the web, relational databases, At the highest level, this is the background processing, and software engineering in general. The enhancements recommended in this chapter are a clear blue path for further improvement and expansion of the system.

References

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2015.

[2] R. Nixon, Learning PHP, MySQL & JavaScript, 6th ed., O'Reilly Media, 2021.

[3] Freshworks Inc., "Freshdesk - Customer Support Software," [Online]. Available: https://freshdesk.com. [Accessed 2025].

[4] Zendesk Inc., "Zendesk - Customer Service Platform," [Online]. Available: https://zendesk.com. [Accessed 2025].

[5] Atlassian, "JIRA Service Management," [Online]. Available: https://atlassian.com/software/jira/service-management. [Accessed 2025].

[6] osTicket, "osTicket - Open Source Support Ticket System," [Online]. Available: https://osticket.com. [Accessed 2025].

[7] PHP Group, "PHP: Hypertext Preprocessor," [Online]. Available: https://www.php.net. [Accessed 2025].

[8] Oracle Corporation, "MySQL Database," [Online]. Available: https://www.mysql.com. [Accessed 2025].

[9] MariaDB Foundation, "MariaDB Server," [Online]. Available: https://mariadb.org. [Accessed 2025].

[10] Tailwind Labs, "Tailwind CSS - Utility First CSS Framework," [Online]. Available: https://tailwindcss.com. [Accessed 2025].

[11] Apache Friends, "XAMPP - Apache + MariaDB + PHP," [Online]. Available: https://www.apachefriends.org. [Accessed 2025].

[12] Microsoft, "Visual Studio Code," [Online]. Available: https://code.visualstudio.com. [Accessed 2025].

[13] PHPMailer, "PHPMailer - A full-featured email creation and transfer class for PHP," [Online]. Available: https://github.com/PHPMailer/PHPMailer. [Accessed 2025].

[14] B. Boehm, "A Spiral Model of Software Development and Enhancement," IEEE Computer, vol. 21, no. 5, pp. 61-72, 1988.

[15] I. Sommerville, Software Engineering, 10th ed., Pearson, 2015.

[16] P. Chen, "The Entity-Relationship Model: Toward a Unified View of Data," ACM Transactions on Database Systems, vol. 1, no. 1, pp. 9-36, 1976.

[17] A. Gemino and D. Parker, "Use Case Diagrams in Support of Use Case Modeling," Journal of Database Management, vol. 20, no. 1, pp. 1-24, 2009.

[18] OWASP Foundation, "OWASP Top Ten Web Application Security Risks," [Online]. Available: https://owasp.org/www-project-top-ten/. [Accessed 2025].

[19] J. Duckett, PHP & MySQL: Server-Side Web Development, Wiley, 2022.

[20] MDN Web Docs, "HTTP Session Management," [Online]. Available: https://developer.mozilla.org. [Accessed 2025].

Appendices

Appendix A

- Using this application step by step

Appendix B

- Code Snippets

Appendix C

- About Supervisor
- About Developer

Appendix A: A step-by-step guide to use this application

1. Accessing the Application

- Go to a web browser, and browse to the UniResolve URL (e.g., http://localhost/uniresolve/index.php). The Login Page will appear.
- Enter your Registered e-mail address and password to log in. The system will Automatically provide you with a student, staff or admin dashboard depending on your role..

2. Submitting a Complaint (Student)

- From the sidebar, click on "Submit Complaint".
- Click on Complaint Type (Academic, Hostel, IT etc) and select if it is a Academic/Hostel/IT Complaint etc. "Individual" or "Collective".
- Fill out the Title and Description and, if necessary, include any supporting documents (PDF, Word, etc.). JPG, PNG).
- Click "Submit". Each person will be assigned a unique Tracking ID (CMP-2026-CS-XXX) for your complaint.

3. Tracking and Communication

- Use the "Track Complaint" page to see the status of your complaint in real-time through the use of the Tracking ID.
- If you have any comments or feedback, use the Comment/Feedback section at the bottom of the tracking page.

4. Staff and Admin Operations

- Directly with the staff member assigned. Responsibilities in staff and admin operations. Completed), and access their corresponding cases.Staff can see their complaints, mark their status (Pending, In Progress, Completed), and access their respective complaints. Make private internal notes and insert the following: Resolved).
- Administrators can manage users, establish routing rules for automatic assignment, view system-wide analytics.

Appendix B: Code Snippets

Name	Registration Number
Muhammad Saddique	FA22-BCS-053
Asad Haroon	FA22-BCS-004
Shahzaib Khurram	FA22-BCS-027

1. Role-based Authentication Routing (index.php)

```
<?php
session_start();
require_once 'includes/functions.php';

if (isset($_SESSION['user_id'])) {
    $base = get_base_url();
    if ($_SESSION['user_type'] === 'student') redirect($base .
'/modules/student/dashboard.php');
    elseif ($_SESSION['user_type'] === 'staff') redirect($base .
'/modules/staff/dashboard.php');
    elseif ($_SESSION['user_type'] === 'admin') redirect($base .
'/modules/admin/dashboard.php');
} else {
    redirect(get_base_url() . '/modules/auth/login.php');
}
?>
```

2. Public Complaint Tracking Query (track.php)

```
// Search for complaint without requiring authentication
$stmt = $pdo->prepare("
    SELECT c.tracking_id, c.current_status, c.created_at, c.response_due_at,
ct.type_name,
           (SELECT priority_value FROM complaint_priority
            WHERE complaint_id = c.complaint_id ORDER BY assigned_at DESC
LIMIT 1) as priority_value
    FROM complaints c
    JOIN complaint_types ct ON c.complaint_type_id = ct.complaint_type_id
    WHERE c.tracking_id = ?
");
```

```
$stmt->execute([$tracking_id]);
$complaint = $stmt->fetch();
```

3. Asynchronous Email Queue Processor - Optimistic Locking (process_queue.php)

```
foreach ($emails as $email) {
    $queue_id = $email['queue_id'];

    // Optimistic locking: mark as sending & increment attempt count
immediately
    // to prevent double-sending under concurrency
    $pdo->prepare("UPDATE email_queue SET status = 'sending', attempts =
attempts + 1
                   WHERE queue_id = ?")->execute([$queue_id]);

    // ... PHPMailer setup and sending logic ...

    // Success update
    $pdo->prepare("UPDATE email_queue SET status = 'sent', sent_at =
CURRENT_TIMESTAMP,
                   error_message = NULL WHERE queue_id = ?")-
>execute([$queue_id]);
}
```

Appendix C: About Developers and Supervisor

The UniResolve was developed by undergraduate students pursuing a Bachelor of Science in Computer Science (BSCS) at the Mirpur University of Science and Technology (MUST), Session 2022-2026.

Name	Registration Number
Muhammad Saddique	FA22-BCS-053
Asad Haroon	FA22-BCS-004
Shahzaib Khurram	FA22-BCS-027

About Supervisors



Dr Faisal Riaz

Professor

faisal.riaz@must.edu.pk

director.oric@must.edu.pk

Prof. Dr. Faisal Riaz is a Professor at Mirpur University of Science and Technology (MUST), Mirpur, Azad Jammu and Kashmir, and serves as the Head of the Department of Computer Science & Information Technology (CS & IT). He is also associated with the Office of Research, Innovation and Commercialization (ORIC) at MUST and serves as Dean of the Faculty of Health & Medical Sciences. He received his PhD in Cyber-Physical Systems (CPS) from Iqra University, Pakistan, in 2018. His research interests include autonomous vehicles, vehicular cyber-physical systems, cognitive radio, affective computing, agent-based modelling, machine learning, and robotics. He heads the Control, Automotive and Robotics (CAR) Laboratory at MUST, where his research focuses on developing intelligent systems and autonomous technologies. He is the author of numerous publications Published in national and international journals and conferences and contributed to research in The high-value, innovative research areas include autonomous navigation, vehicle-to-vehicle communication and intelligent transport. transportation systems. His academic and research work have been recognized, Including this year's Hall of Fame for Smart Innovations in 2018.



Mr. Mohsin Ansari

Lecturer

mohsen.csit@must.edu.pk

Muhammad Mohsan Ansari is the Lecturer of Mirpur University of Science and Technology. This is in reference to the Motor Solutions and Transport (MUST) of Mirpur in the Azad Jammu and Kashmir. He has been linked to the university's academic activities in the field of Computer Science and Information Uses technology and contributes to the academic growth of students and teachers. As a faculty member, he is involved in supporting students in their academic and technical work and supports the mission of the university in education. He is a lecturer in... his role in delivering higher education and guiding students in their studies and academic projects.

About Developers



Muhammad Saddique

FA22-BCS-053

siddiquerajput1028@gmail.com

Muhammad Saddique is a Final year BS Computer Science Student at Mirpur University of Technology. The Science and Technology (MUST), Mirpur, AJK is ranked **3.8/4.0 CGPA.** His academic interests cover: **Artificial Intelligence, Machine Learning, Software Development, Web Development, and Database Systems**. He has created projects such as, **UniResolve – Online Complaint Management System, a Petroleum Tax Management Website, and an Android Banking Application**. He demonstrates realism in his work through his ability to: Programming, Database Management, Software Development.

Asad Haroon

FA22-BCS-004

asadharoon316@gmail.com

Asad Haroon is a student of the final year of BS Computer Science in Mirpur University of Science. Mirpur, AJK - and Technology (MUST). His academic research interests are in **Artificial Intelligence, Pattern Recognition, and Robotics. Intelligence, Machine Learning, Web Development and Database Systems**. He has Went to work for the government on projects such as **UniResolve – Online Complaint Management System along with a Dogri Machine Translation System**. practical skills of his work are shown in his work. Programming, database management, Web development and AI applications.

Shahzaib Khurram

FA22-BCS-027

shahzaibkhurram25@gmail.com

Shahzaib Khurram is a Computer Science Student who has knowledge and experience in Web and Application Development. development. He is passionate about **Artificial Intelligence and Machine learning** and has an interest in them. how they can be used to solve real-world problems. He is an **alumni of Harvard's CS50x: Introduction to Computer Science,** which reinforced his algorithmic and data skills, Structures, Software Engineering. He is dedicated to creating innovative and scalable Solutions using modern Web technologies and intelligent systems.